

1. Clones PlantDoc dataset
2. Checks dataset structure
3. Removes corrupted images
4. Creates train, validation, and test records
5. Applies data augmentation
6. Trains 5 models
7. Fine-tunes transfer learning models
8. Evaluates accuracy, precision, recall, F1-score
9. Calculates model size
10. Calculates training time
11. Calculates inference time per image
12. Generates confusion matrix
13. Generates comparison charts
14. Creates paper-ready result tables
15. Creates ZIP file of all results

PlantDoc introduced a real-world plant disease dataset, but detailed comparison of modern transfer learning models using both accuracy and practical deployment metrics was limited.

Therefore, this study compares Custom CNN, MobileNetV2, ResNet50, EfficientNetB0, and DenseNet121 on PlantDoc using accuracy, precision, recall, F1-score, model size, training time, and inference time.

Comparative Deep Learning Analysis for Real- World Plant Disease Detection Using PlantDoc Dataset

This notebook compares five deep learning models on the PlantDoc dataset:

1. Custom CNN
2. MobileNetV2
3. ResNet50
4. EfficientNetB0
5. DenseNet121

Evaluation metrics:

- Accuracy
- Precision

- Recall
- F1-score
- Model size
- Training time
- Inference time per image

The objective is to identify both the best-performing model and the most practical model for low-resource or mobile deployment.

```
# =====
# STEP 1: INSTALL AND IMPORT REQUIRED LIBRARIES
# =====

!pip install tensorflow

import os
import time
import shutil
import random
import zipfile
import gc
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from PIL import Image

import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import classification_report, confusion_matrix,
from sklearn.utils.class_weight import compute_class_weight

print("TensorFlow Version:", tf.__version__)
print("GPU Available:", tf.config.list_physical_devices("GPU"))
```

```
Requirement already satisfied: tensorflow in /usr/local/lib/python3.
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/pyth
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/p
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/li
Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 i
Requirement already satisfied: google_pasta>=0.1.1 in /usr/local/lib
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/py
Requirement already satisfied: opt_einsum>=2.3.2 in /usr/local/lib/p
Requirement already satisfied: packaging in /usr/local/lib/python3.1
Requirement already satisfied: protobuf>=5.28.0 in /usr/local/lib/py
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib
```

```

Requirement already satisfied: setuptools in /usr/local/lib/python3.
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/py
Requirement already satisfied: typing_extensions>=3.6.6 in /usr/loca
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/pythc
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib
Requirement already satisfied: tensorboard~=2.20.0 in /usr/local/lib
Requirement already satisfied: keras>=3.10.0 in /usr/local/lib/pythc
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/pythc
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/pythor
Requirement already satisfied: ml_dtypes<1.0.0,>=0.5.1 in /usr/local
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/
Requirement already satisfied: rich in /usr/local/lib/python3.12/dis
Requirement already satisfied: namex in /usr/local/lib/python3.12/di
Requirement already satisfied: optree in /usr/local/lib/python3.12/c
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/loca
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/pythor
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/pyt
Requirement already satisfied: pillow in /usr/local/lib/python3.12/c
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/pyt
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/p
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/l
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.
TensorFlow Version: 2.20.0
GPU Available: []

```

```

# =====
# STEP 2: BASIC CONFIGURATION
# =====

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)

IMG_SIZE = (224, 224)
BATCH_SIZE = 16

# For quick test, use fewer epochs.
# For final paper result, increase epochs.
CUSTOM_CNN_EPOCHS = 25
TRANSFER_INITIAL_EPOCHS = 10
TRANSFER_FINE_TUNE_EPOCHS = 10

RESULT_DIR = Path("/content/plantdoc_results")
MODEL_DIR = RESULT_DIR / "saved_models"
FIGURE_DIR = RESULT_DIR / "figures"

RESULT_DIR.mkdir(exist_ok=True)
MODEL_DIR.mkdir(exist_ok=True)
FIGURE_DIR.mkdir(exist_ok=True)

```

```
print("Image size:", IMG_SIZE)
print("Batch size:", BATCH_SIZE)
print("Result directory:", RESULT_DIR)
```

```
Image size: (224, 224)
Batch size: 16
Result directory: /content/plantdoc_results
```

```
# =====
# STEP 3: DOWNLOAD PLANTDOC DATASET
# =====

# This is the public PlantDoc dataset repository.
# If this clone fails, manually upload dataset ZIP and extract it.

DATASET_REPO = "https://github.com/pratikkayal/PlantDoc-Dataset.git"
DATASET_DIR = Path("/content/PlantDoc-Dataset")

if not DATASET_DIR.exists():
    !git clone https://github.com/pratikkayal/PlantDoc-Dataset.git
else:
    print("Dataset already exists.")

print("Dataset downloaded/existing at:", DATASET_DIR)
```

```
Cloning into '/content/PlantDoc-Dataset'...
remote: Enumerating objects: 2670, done.
remote: Counting objects: 100% (35/35), done.
remote: Compressing objects: 100% (13/13), done.
remote: Total 2670 (delta 22), reused 22 (delta 22), pack-reused 263
Receiving objects: 100% (2670/2670), 932.92 MiB | 24.37 MiB/s, done.
Resolving deltas: 100% (24/24), done.
Updating files: 100% (2581/2581), done.
Dataset downloaded/existing at: /content/PlantDoc-Dataset
```

Start coding or [generate](#) with AI.

```
# =====
# STEP 4: FIND IMAGE FILES AND CLASS LABELS
# =====

image_extensions = {".jpg", ".jpeg", ".png", ".bmp", ".webp"}

def find_image_files(dataset_root):
    dataset_root = Path(dataset_root)
    records = []

    for img_path in dataset_root.rglob("*"):
        if img_path.suffix.lower() in image_extensions:
            label = img_path.parent.name.strip()
            records.append({
                "filepath": str(img_path),
                "label": label
            })
```

```

    )

    return pd.DataFrame(records)

df = find_image_files(DATASET_DIR)

print("Total image files found:", len(df))
print("Sample records:")
display(df.head())

print("\nNumber of raw classes:", df["label"].nunique())
print("\nClass distribution:")
display(df["label"].value_counts().head(40))

```

Total image files found: 2579

Sample records:

	filepath	label	
0	/content/PlantDoc-Dataset/PlantDoc_Examples.png	PlantDoc-Dataset	
1	/content/PlantDoc-Dataset/train/grape leaf bla...	grape leaf black rot	
2	/content/PlantDoc-Dataset/train/grape leaf bla...	grape leaf black rot	
3	/content/PlantDoc-Dataset/train/grape leaf bla...	grape leaf black rot	
4	/content/PlantDoc-Dataset/train/grape leaf bla...	grape leaf black rot	

Number of raw classes: 29

Class distribution:

	count
label	
Corn leaf blight	192
Tomato Septoria leaf spot	151
Squash Powdery mildew leaf	130
Raspberry leaf	119
Blueberry leaf	117
Potato leaf early blight	117
Corn rust leaf	116
Peach leaf	112
Tomato leaf late blight	111
Tomato leaf bacterial spot	110
Potato leaf late blight	105
Strawberry leaf	96
Apple Scab Leaf	93

Tomato mold leaf	91
Apple leaf	91
Apple rust leaf	89
Tomato Early blight leaf	88
Tomato leaf yellow virus	76
Bell_pepper leaf spot	71
grape leaf	69
Corn Gray leaf spot	68
Soyabean leaf	65
grape leaf black rot	64
Tomato leaf	63
Bell_pepper leaf	61
Cherry leaf	57
Tomato leaf mosaic virus	54
Tomato two spotted spider mites leaf	2
PlantDoc-Dataset	1

dtype: int64

```
# =====
# STEP 5: REMOVE CORRUPTED OR UNREADABLE IMAGES
# =====

def is_valid_image(path):
    try:
        with Image.open(path) as img:
            img.verify()
        return True
    except Exception:
        return False

valid_flags = []

for path in df["filepath"]:
    valid_flags.append(is_valid_image(path))

df["valid"] = valid_flags

invalid_count = (~df["valid"]).sum()
print("Invalid images found:", invalid_count)

df = df[df["valid"]].drop(columns=["valid"]).reset_index(drop=True)

print("Images after cleaning:", len(df))
```

```
print( images after cleaning: , len(df))  
print("Classes after cleaning:", df["label"].nunique())
```

```
Invalid images found: 0  
Images after cleaning: 2579  
Classes after cleaning: 29
```

```
# =====  
# STEP 6: FILTER CLASSES WITH VERY FEW IMAGES  
# =====  
  
# Keep classes with at least 10 images.  
# You can increase this to 20 or 50 for stricter training.  
MIN_IMAGES_PER_CLASS = 10  
  
class_counts = df["label"].value_counts()  
valid_classes = class_counts[class_counts >= MIN_IMAGES_PER_CLASS].  
  
df = df[df["label"].isin(valid_classes)].reset_index(drop=True)  
  
print("Images after class filtering:", len(df))  
print("Final number of classes:", df["label"].nunique())  
  
display(df["label"].value_counts())
```

```
Images after class filtering: 2576  
Final number of classes: 27
```

	count
label	
Corn leaf blight	192
Tomato Septoria leaf spot	151
Squash Powdery mildew leaf	130
Raspberry leaf	119
Potato leaf early blight	117
Blueberry leaf	117
Corn rust leaf	116
Peach leaf	112
Tomato leaf late blight	111
Tomato leaf bacterial spot	110
Potato leaf late blight	105
Strawberry leaf	96

Apple Scab Leaf	93
Tomato mold leaf	91
Apple leaf	91
Apple rust leaf	89
Tomato Early blight leaf	88
Tomato leaf yellow virus	76
Bell_pepper leaf spot	71
grape leaf	69
Corn Gray leaf spot	68
Soyabean leaf	65
grape leaf black rot	64
Tomato leaf	63
Bell_pepper leaf	61
Cherry leaf	57
Tomato leaf mosaic virus	54

dtype: int64

```
# =====
# STEP 7: ENCODE TEXT LABELS INTO NUMERIC LABELS
# =====

label_encoder = LabelEncoder()
df["label_id"] = label_encoder.fit_transform(df["label"])

class_names = list(label_encoder.classes_)
num_classes = len(class_names)

print("Number of classes:", num_classes)
print("Class names:")
for i, name in enumerate(class_names):
    print(i, ":", name)
```

```
Number of classes: 27
Class names:
0 : Apple Scab Leaf
1 : Apple leaf
2 : Apple rust leaf
3 : Bell pepper leaf
```

```

3 : Bell_pepper leaf
4 : Bell_pepper leaf spot
5 : Blueberry leaf
6 : Cherry leaf
7 : Corn Gray leaf spot
8 : Corn leaf blight
9 : Corn rust leaf
10 : Peach leaf
11 : Potato leaf early blight
12 : Potato leaf late blight
13 : Raspberry leaf
14 : Soyabean leaf
15 : Squash Powdery mildew leaf
16 : Strawberry leaf
17 : Tomato Early blight leaf
18 : Tomato Septoria leaf spot
19 : Tomato leaf
20 : Tomato leaf bacterial spot
21 : Tomato leaf late blight
22 : Tomato leaf mosaic virus
23 : Tomato leaf yellow virus
24 : Tomato mold leaf
25 : grape leaf
26 : grape leaf black rot

```

Start coding or [generate](#) with AI.

```

# =====
# STEP 8: CREATE TRAIN, VALIDATION, AND TEST SPLITS
# =====

# Split:
# 70% training
# 15% validation
# 15% testing

train_df, temp_df = train_test_split(
    df,
    test_size=0.30,
    random_state=SEED,
    stratify=df["label_id"]
)

val_df, test_df = train_test_split(
    temp_df,
    test_size=0.50,
    random_state=SEED,
    stratify=temp_df["label_id"]
)

train_df = train_df.reset_index(drop=True)
val_df = val_df.reset_index(drop=True)
test_df = test_df.reset_index(drop=True)

print("Training images:", len(train_df))
print("Validation images:", len(val_df))

```

```
print( validation_images, len(val_df))
print("Testing images:", len(test_df))

print("\nTrain class distribution:")
display(train_df["label"].value_counts().head())
```

Training images: 1803
 Validation images: 386
 Testing images: 387

Train class distribution:

	count
label	
Corn leaf blight	134
Tomato Septoria leaf spot	106
Squash Powdery mildew leaf	91
Raspberry leaf	83
Blueberry leaf	82

dtype: int64

```
# =====
# STEP 9: COMPUTE CLASS WEIGHTS
# =====

classes = np.unique(train_df["label_id"])

class_weights_array = compute_class_weight(
    class_weight="balanced",
    classes=classes,
    y=train_df["label_id"]
)

class_weights = {
    int(cls): float(weight)
    for cls, weight in zip(classes, class_weights_array)
}

print("Class weights calculated.")
```

Class weights calculated.

```
# =====
# STEP 10: DATA AUGMENTATION AND PREPROCESSING
# =====

data_augmentation = tf.keras.Sequential([
```

```
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.10),
    layers.RandomZoom(0.10),
    layers.RandomContrast(0.10),
], name="data_augmentation")

def get_preprocess_function(model_name):
    if model_name == "CustomCNN":
        return lambda x: x / 255.0

    elif model_name == "MobileNetV2":
        return tf.keras.applications.mobilenet_v2.preprocess_input

    elif model_name == "ResNet50":
        return tf.keras.applications.resnet50.preprocess_input

    elif model_name == "EfficientNetB0":
        return tf.keras.applications.efficientnet.preprocess_input

    elif model_name == "DenseNet121":
        return tf.keras.applications.densenet.preprocess_input

    else:
        raise ValueError("Unknown model name")

def load_image(path, label):
    image = tf.io.read_file(path)
    image = tf.io.decode_image(image, channels=3, expand_animations=False)
    image.set_shape([None, None, 3])
    image = tf.image.resize(image, IMG_SIZE)
    image = tf.cast(image, tf.float32)
    return image, label

def make_dataset(dataframe, model_name, training=False):
    paths = dataframe["filepath"].values
    labels = dataframe["label_id"].values.astype(np.int32)

    preprocess_fn = get_preprocess_function(model_name)

    ds = tf.data.Dataset.from_tensor_slices((paths, labels))

    if training:
        ds = ds.shuffle(buffer_size=len(dataframe), seed=SEED)

    ds = ds.map(load_image, num_parallel_calls=tf.data.AUTOTUNE)

    def augment_and_preprocess(image, label):
        if training:
            image = data_augmentation(image, training=True)

        image = preprocess_fn(image)
```

```
        image = preprocess_fn(image),
        return image, label

ds = ds.map(augment_and_preprocess, num_parallel_calls=tf.data.
ds = ds.batch(BATCH_SIZE)
ds = ds.prefetch(tf.data.AUTOTUNE)

return ds
```

```
# =====
# STEP 11: CUSTOM CNN MODEL
# =====

def build_custom_cnn(num_classes):
    model = models.Sequential([
        layers.Input(shape=(224, 224, 3)),

        layers.Conv2D(32, (3, 3), padding="same", activation="relu"
        layers.BatchNormalization(),
        layers.MaxPooling2D(),

        layers.Conv2D(64, (3, 3), padding="same", activation="relu"
        layers.BatchNormalization(),
        layers.MaxPooling2D(),

        layers.Conv2D(128, (3, 3), padding="same", activation="relu"
        layers.BatchNormalization(),
        layers.MaxPooling2D(),

        layers.Conv2D(256, (3, 3), padding="same", activation="relu"
        layers.BatchNormalization(),
        layers.MaxPooling2D(),

        layers.GlobalAveragePooling2D(),

        layers.Dense(256, activation="relu"),
        layers.Dropout(0.5),

        layers.Dense(num_classes, activation="softmax")
    ], name="CustomCNN")

    return model
```

```
# =====
# STEP 12: TRANSFER LEARNING MODEL BUILDERS
# =====

def build_transfer_model(model_name, num_classes):
    input_shape = (224, 224, 3)

    if model_name == "MobileNetV2":
        base_model = tf.keras.applications.MobileNetV2(
            include_top=False,
            weights="imagenet",
            input_shape=input_shape
        )

    elif model_name == "ResNet50":
        base_model = tf.keras.applications.ResNet50(
            include_top=False,
            weights="imagenet",
            input_shape=input_shape
        )

    elif model_name == "EfficientNetB0":
        base_model = tf.keras.applications.EfficientNetB0(
            include_top=False,
            weights="imagenet",
            input_shape=input_shape
        )

    elif model_name == "DenseNet121":
        base_model = tf.keras.applications.DenseNet121(
            include_top=False,
            weights="imagenet",
            input_shape=input_shape
        )

    else:
        raise ValueError("Unknown transfer model name")

    base_model.trainable = False

    inputs = layers.Input(shape=input_shape)
    x = base_model(inputs, training=False)
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.BatchNormalization()(x)
    x = layers.Dropout(0.4)(x)
    x = layers.Dense(256, activation="relu")(x)
```

```
x = layers.Dropout(0.3)(x)
outputs = layers.Dense(num_classes, activation="softmax")(x)

model = models.Model(inputs, outputs, name=model_name)

return model, base_model
```

```
# =====
# STEP 13: TRAINING AND EVALUATION FUNCTIONS
# =====

def compile_model(model, learning_rate):
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=learning_r
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )

def get_callbacks(model_name):
    checkpoint_path = MODEL_DIR / f"{model_name}_best.keras"

    callbacks = [
        EarlyStopping(
            monitor="val_accuracy",
            patience=5,
            restore_best_weights=True,
            verbose=1
        ),
        ReduceLROnPlateau(
            monitor="val_loss",
            factor=0.3,
            patience=3,
            min_lr=1e-7,
            verbose=1
        ),
        ModelCheckpoint(
            filepath=str(checkpoint_path),
            monitor="val_accuracy",
            save_best_only=True,
            verbose=1
        )
    ]

    return callbacks
```

```
def fine_tune_base_model(base_model, fine_tune_at=None):
    base_model.trainable = True

    if fine_tune_at is None:
        fine_tune_at = int(len(base_model.layers) * 0.70)

    for layer in base_model.layers[:fine_tune_at]:
        layer.trainable = False

    # Keep BatchNormalization layers frozen for stable fine-tuning
    for layer in base_model.layers:
        if isinstance(layer, layers.BatchNormalization):
            layer.trainable = False

def evaluate_model(model, test_ds, model_name):
    y_true = test_df["label_id"].values

    start_time = time.time()
    y_prob = model.predict(test_ds, verbose=1)
    total_inference_time = time.time() - start_time

    y_pred = np.argmax(y_prob, axis=1)

    report = classification_report(
        y_true,
        y_pred,
        target_names=class_names,
        output_dict=True,
        zero_division=0
    )

    accuracy = accuracy_score(y_true, y_pred)
    precision_macro = report["macro avg"]["precision"]
    recall_macro = report["macro avg"]["recall"]
    f1_macro = report["macro avg"]["f1-score"]

    inference_time_per_image = total_inference_time / len(test_df)

    return {
        "Model": model_name,
        "Accuracy": accuracy,
        "Precision_Macro": precision_macro,
        "Recall_Macro": recall_macro,
        "F1_Macro": f1_macro,
        "Total_Inference_Time_sec": total_inference_time,
        "Inference_Time_per_Image_sec": inference_time_per_image,
        "y_true": y_true,
        "y_pred": y_pred,
        "classification_report": report
    }
```



```
# =====
# STEP 14: CONFUSION MATRIX FUNCTION
# =====

def plot_confusion_matrix(y_true, y_pred, model_name):
    cm = confusion_matrix(y_true, y_pred)

    plt.figure(figsize=(18, 16))
    plt.imshow(cm, interpolation="nearest")
    plt.title(f"Confusion Matrix - {model_name}", fontsize=16)
    plt.colorbar()

    tick_marks = np.arange(len(class_names))
    plt.xticks(tick_marks, class_names, rotation=90, fontsize=8)
    plt.yticks(tick_marks, class_names, fontsize=8)

    plt.xlabel("Predicted Label")
    plt.ylabel("True Label")
    plt.tight_layout()

    save_path = FIGURE_DIR / f"{model_name}_confusion_matrix.png"
    plt.savefig(save_path, dpi=300, bbox_inches="tight")
    plt.show()

    print("Saved:", save_path)
```

```
# =====
# STEP 15: COMPLETE TRAINING PIPELINE FOR ONE MODEL
# =====

def train_and_evaluate_model(model_name):
    print("\n" + "=" * 80)
    print("Training Model:", model_name)
    print("=" * 80)

    train_ds = make_dataset(train_df, model_name, training=True)
    val_ds = make_dataset(val_df, model_name, training=False)
    test_ds = make_dataset(test_df, model_name, training=False)

    if model_name == "CustomCNN":
        model = build_custom_cnn(num_classes)
```

```

base_model = None

compile_model(model, learning_rate=1e-3)

start_train_time = time.time()

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=CUSTOM_CNN_EPOCHS,
    class_weight=class_weights,
    callbacks=get_callbacks(model_name),
    verbose=1
)

training_time = time.time() - start_train_time

else:
    model, base_model = build_transfer_model(model_name, num_cl

    compile_model(model, learning_rate=1e-3)

    start_train_time = time.time()

    print("\nInitial training with frozen base model...")

    history_initial = model.fit(
        train_ds,
        validation_data=val_ds,
        epochs=TRANSFER_INITIAL_EPOCHS,
        class_weight=class_weights,
        callbacks=get_callbacks(model_name + "_initial"),
        verbose=1
    )

    print("\nFine-tuning selected layers...")

    fine_tune_base_model(base_model)

    compile_model(model, learning_rate=1e-5)

    history_fine = model.fit(
        train_ds,
        validation_data=val_ds,
        epochs=TRANSFER_FINE_TUNE_EPOCHS,
        class_weight=class_weights,
        callbacks=get_callbacks(model_name + "_finetune"),
        verbose=1
    )

    training_time = time.time() - start_train_time

    # combine history manually
    history = {
        "train_loss": history_initial.history["train_loss"] + history_fine.history["train_loss"],
        "val_loss": history_initial.history["val_loss"] + history_fine.history["val_loss"],
        "train_acc": history_initial.history["train_acc"] + history_fine.history["train_acc"],
        "val_acc": history_initial.history["val_acc"] + history_fine.history["val_acc"],
    }

```

```

        "initial": nistory_initial.nistory,
        "fine_tune": history_fine.history
    }

# Save final model
final_model_path = MODEL_DIR / f"{model_name}_final.keras"
model.save(final_model_path)

model_size_mb = os.path.getsize(final_model_path) / (1024 * 1024)
total_params = model.count_params()

eval_result = evaluate_model(model, test_ds, model_name)

eval_result["Training_Time_sec"] = training_time
eval_result["Training_Time_min"] = training_time / 60
eval_result["Model_Size_MB"] = model_size_mb
eval_result["Total_Parameters"] = total_params
eval_result["Saved_Model_Path"] = str(final_model_path)

print("\nEvaluation Summary:")
print("Accuracy:", eval_result["Accuracy"])
print("Precision Macro:", eval_result["Precision_Macro"])
print("Recall Macro:", eval_result["Recall_Macro"])
print("F1 Macro:", eval_result["F1_Macro"])
print("Model Size MB:", eval_result["Model_Size_MB"])
print("Training Time min:", eval_result["Training_Time_min"])
print("Inference Time/Image sec:", eval_result["Inference_Time"])

plot_confusion_matrix(
    eval_result["y_true"],
    eval_result["y_pred"],
    model_name
)

del model
gc.collect()
tf.keras.backend.clear_session()

return eval_result

```

```

# =====
# STEP 16: TRAIN AND EVALUATE ALL FIVE MODELS
# =====

models_to_train = [
    "CustomCNN".

```

```

        "MobileNetV2",
        "ResNet50",
        "EfficientNetB0",
        "DenseNet121"
    ]

    all_results = []

    for model_name in models_to_train:
        result = train_and_evaluate_model(model_name)
        all_results.append(result)

    print("All models trained and evaluated.")

```

```

=====
Training Model: CustomCNN
=====

```

```
Epoch 1/25
```

```
113/113 ----- 0s 4s/step - accuracy: 0.0593 - loss: 3.6553
```

```
Epoch 1: val_accuracy improved from None to 0.03627, saving model to /content/plantdoc_results/saved_models/CustomCNN_best.keras
```

```
Epoch 1: finished saving model to /content/plantdoc_results/saved_models/CustomCNN_best.keras
```

```
113/113 ----- 442s 4s/step - accuracy: 0.0682 - loss: 3.47
```

```
Epoch 2/25
```

```
113/113 ----- 0s 4s/step - accuracy: 0.0918 - loss: 3.2483
```

```
Epoch 2: val_accuracy improved from 0.03627 to 0.04663, saving model to /content/plantdoc_results/saved_models/CustomCNN_best.keras
```

```
Epoch 2: finished saving model to /content/plantdoc_results/saved_models/CustomCNN_best.keras
```

```
113/113 ----- 442s 4s/step - accuracy: 0.0871 - loss: 3.24
```

```
Epoch 3/25
```

```
113/113 ----- 0s 4s/step - accuracy: 0.1080 - loss: 3.1673
```

```
Epoch 3: val_accuracy improved from 0.04663 to 0.06218, saving model to /content/plantdoc_results/saved_models/CustomCNN_best.keras
```

```
Epoch 3: finished saving model to /content/plantdoc_results/saved_models/CustomCNN_best.keras
```

```
113/113 ----- 423s 4s/step - accuracy: 0.1026 - loss: 3.16
```

```
Epoch 4/25
```

```
113/113 ----- 0s 3s/step - accuracy: 0.1146 - loss: 3.1294
```

```
Epoch 4: val_accuracy improved from 0.06218 to 0.10881, saving model to /content/plantdoc_results/saved_models/CustomCNN_best.keras
```

```
Epoch 4: finished saving model to /content/plantdoc_results/saved_models/CustomCNN_best.keras
```

```
113/113 ----- 438s 4s/step - accuracy: 0.1109 - loss: 3.10
```

```
Epoch 5/25
```

```
113/113 ----- 0s 4s/step - accuracy: 0.1155 - loss: 3.0577
```

```
Epoch 5: val_accuracy improved from 0.10881 to 0.15803, saving model to /content/plantdoc_results/saved_models/CustomCNN_best.keras
```

```
Epoch 5: finished saving model to /content/plantdoc_results/saved_models/CustomCNN_best.keras
```

```
113/113 ----- 456s 4s/step - accuracy: 0.1237 - loss: 3.04
```

```
Epoch 6/25
```

```
113/113 ----- 0s 4s/step - accuracy: 0.1539 - loss: 2.9956
```

```
Epoch 6: val_accuracy did not improve from 0.15803
```

```
113/113 ----- 419s 4s/step - accuracy: 0.1453 - loss: 3.01
```

```
Epoch 7/25
```

113/113 ————— **0s** 4s/step - accuracy: 0.1390 - loss: 2.9716
Epoch 7: val_accuracy did not improve from 0.15803
113/113 ————— **439s** 4s/step - accuracy: 0.1536 - loss: 2.96
Epoch 8/25
113/113 ————— **0s** 3s/step - accuracy: 0.1587 - loss: 2.8421
Epoch 8: ReduceLROnPlateau reducing learning rate to 0.0003000000142492354.

Epoch 8: val_accuracy did not improve from 0.15803
113/113 ————— **418s** 4s/step - accuracy: 0.1581 - loss: 2.90
Epoch 9/25
113/113 ————— **0s** 3s/step - accuracy: 0.1680 - loss: 2.8314
Epoch 9: val_accuracy did not improve from 0.15803
113/113 ————— **438s** 4s/step - accuracy: 0.1692 - loss: 2.81
Epoch 10/25
113/113 ————— **0s** 3s/step - accuracy: 0.2007 - loss: 2.7557
Epoch 10: val_accuracy improved from 0.15803 to 0.18912, saving model to /content/plantdoc_results/

Epoch 10: finished saving model to /content/plantdoc_results/saved_models/CustomCNN_best.keras
113/113 ————— **436s** 4s/step - accuracy: 0.2069 - loss: 2.73
Epoch 11/25
113/113 ————— **0s** 3s/step - accuracy: 0.2108 - loss: 2.6588
Epoch 11: val_accuracy did not improve from 0.18912
113/113 ————— **437s** 4s/step - accuracy: 0.2024 - loss: 2.67
Epoch 12/25
113/113 ————— **0s** 3s/step - accuracy: 0.2285 - loss: 2.6129
Epoch 12: val_accuracy did not improve from 0.18912
113/113 ————— **417s** 4s/step - accuracy: 0.2219 - loss: 2.64
Epoch 13/25
113/113 ————— **0s** 3s/step - accuracy: 0.2096 - loss: 2.6112
Epoch 13: val_accuracy improved from 0.18912 to 0.20207, saving model to /content/plantdoc_results/

Epoch 13: finished saving model to /content/plantdoc_results/saved_models/CustomCNN_best.keras
113/113 ————— **418s** 4s/step - accuracy: 0.2158 - loss: 2.62
Epoch 14/25
113/113 ————— **0s** 3s/step - accuracy: 0.2355 - loss: 2.5531
Epoch 14: val_accuracy did not improve from 0.20207
113/113 ————— **439s** 4s/step - accuracy: 0.2385 - loss: 2.56
Epoch 15/25
113/113 ————— **0s** 3s/step - accuracy: 0.2586 - loss: 2.5449
Epoch 15: val_accuracy improved from 0.20207 to 0.22021, saving model to /content/plantdoc_results/

Epoch 15: finished saving model to /content/plantdoc_results/saved_models/CustomCNN_best.keras
113/113 ————— **443s** 4s/step - accuracy: 0.2413 - loss: 2.56
Epoch 16/25
113/113 ————— **0s** 3s/step - accuracy: 0.2444 - loss: 2.5441
Epoch 16: val_accuracy did not improve from 0.22021
113/113 ————— **435s** 4s/step - accuracy: 0.2501 - loss: 2.53
Epoch 17/25
113/113 ————— **0s** 3s/step - accuracy: 0.2428 - loss: 2.5126
Epoch 17: val_accuracy did not improve from 0.22021
113/113 ————— **436s** 4s/step - accuracy: 0.2562 - loss: 2.51
Epoch 18/25
113/113 ————— **0s** 3s/step - accuracy: 0.2500 - loss: 2.4820
Epoch 18: ReduceLROnPlateau reducing learning rate to 9.000000427477062e-05.

Epoch 18: val_accuracy did not improve from 0.22021
113/113 ————— **418s** 4s/step - accuracy: 0.2540 - loss: 2.49
Epoch 19/25
113/113 ————— **0s** 3s/step - accuracy: 0.2800 - loss: 2.3885

Epoch 19: val_accuracy did not improve from 0.22021

113/113 ————— **442s** 4s/step - accuracy: 0.2740 - loss: 2.41

Epoch 20/25

113/113 ————— **0s** 3s/step - accuracy: 0.2968 - loss: 2.3517

Epoch 20: val_accuracy did not improve from 0.22021

113/113 ————— **418s** 4s/step - accuracy: 0.2967 - loss: 2.36

Epoch 20: early stopping

Restoring model weights from the end of the best epoch: 15.

25/25 ————— **23s** 874ms/step

Evaluation Summary:

Accuracy: 0.21188630490956073

Precision Macro: 0.2327006999994616

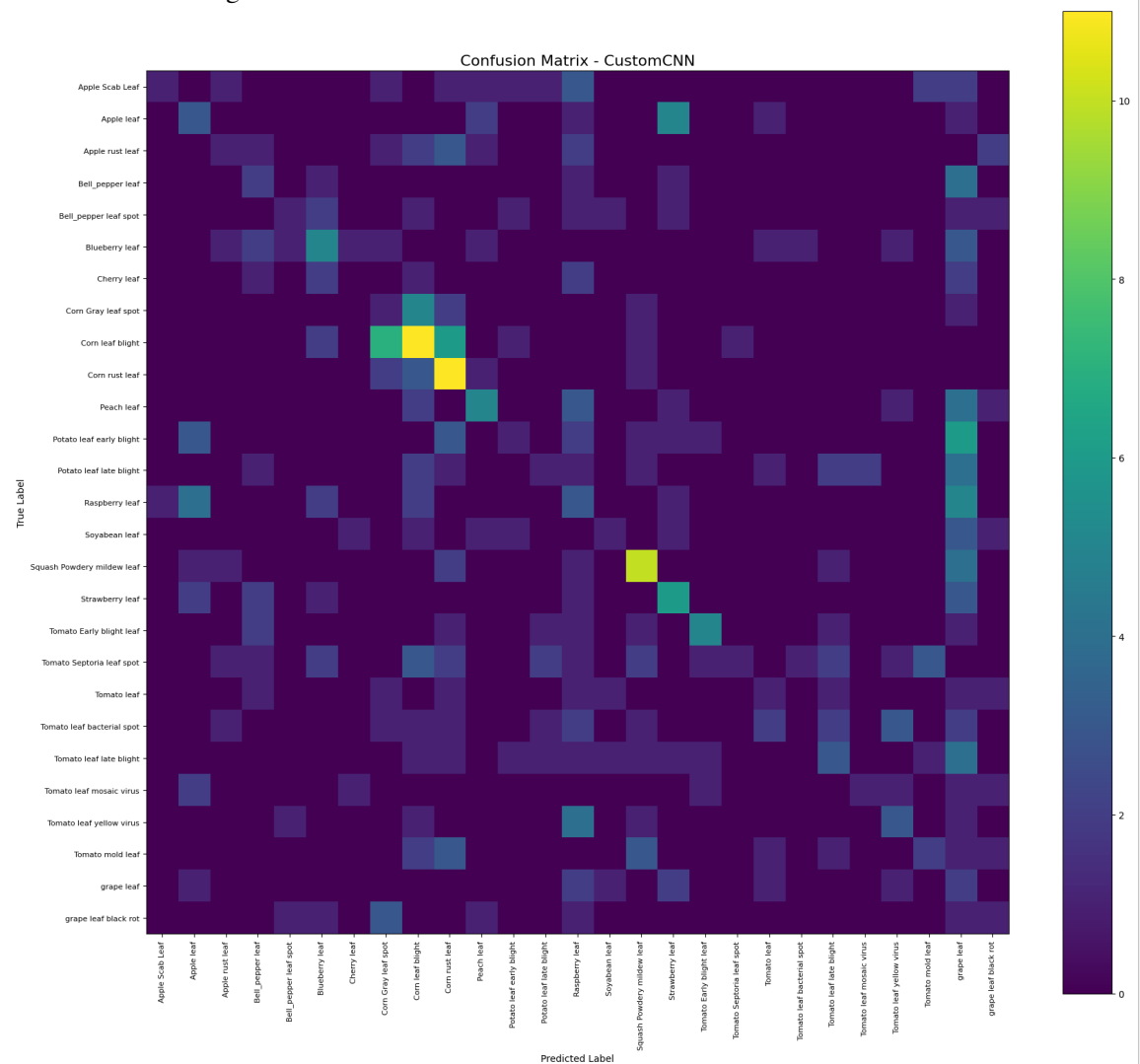
Recall Macro: 0.19317603984694462

F1 Macro: 0.186717378637789

Model Size MB: 5.367273330688477

Training Time min: 144.25997790495555

Inference Time/Image sec: 0.10594047625243509



Saved: /content/plantdoc_results/figures/CustomCNN_confusion_matrix.png

Training Model: MobileNetV2

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_1.0_224.h5

9406464/9406464 ————— **0s** 0us/step

Initial training with frozen base model...

Epoch 1/10

113/113

```

113/113 ————— 0s 984ms/step - accuracy: 0.1266 - loss: 3.2
Epoch 1: val_accuracy improved from None to 0.39896, saving model to /content/plantdoc_results/saved_models/MobileNetV2_initial_best.keras
Epoch 1: finished saving model to /content/plantdoc_results/saved_models/MobileNetV2_initial_best.keras
113/113 ————— 147s 1s/step - accuracy: 0.2041 - loss: 3.30
Epoch 2/10
113/113 ————— 0s 930ms/step - accuracy: 0.3707 - loss: 2.2
Epoch 2: val_accuracy improved from 0.39896 to 0.45337, saving model to /content/plantdoc_results/saved_models/MobileNetV2_initial_best.keras
Epoch 2: finished saving model to /content/plantdoc_results/saved_models/MobileNetV2_initial_best.keras
113/113 ————— 126s 1s/step - accuracy: 0.3894 - loss: 2.22
Epoch 3/10
113/113 ————— 0s 931ms/step - accuracy: 0.4602 - loss: 1.8
Epoch 3: val_accuracy improved from 0.45337 to 0.48446, saving model to /content/plantdoc_results/saved_models/MobileNetV2_initial_best.keras
Epoch 3: finished saving model to /content/plantdoc_results/saved_models/MobileNetV2_initial_best.keras
113/113 ————— 125s 1s/step - accuracy: 0.4653 - loss: 1.84
Epoch 4/10
113/113 ————— 0s 926ms/step - accuracy: 0.5011 - loss: 1.6
Epoch 4: val_accuracy did not improve from 0.48446
113/113 ————— 141s 1s/step - accuracy: 0.5036 - loss: 1.62
Epoch 5/10
113/113 ————— 0s 1s/step - accuracy: 0.5386 - loss: 1.5199
Epoch 5: val_accuracy did not improve from 0.48446
113/113 ————— 135s 1s/step - accuracy: 0.5552 - loss: 1.54
Epoch 6/10
113/113 ————— 0s 934ms/step - accuracy: 0.5670 - loss: 1.4
Epoch 6: ReduceLROnPlateau reducing learning rate to 0.0003000000142492354.

Epoch 6: val_accuracy improved from 0.48446 to 0.49741, saving model to /content/plantdoc_results/saved_models/MobileNetV2_initial_best.keras
Epoch 6: finished saving model to /content/plantdoc_results/saved_models/MobileNetV2_initial_best.keras
113/113 ————— 134s 1s/step - accuracy: 0.5663 - loss: 1.43
Epoch 7/10
113/113 ————— 0s 936ms/step - accuracy: 0.6007 - loss: 1.2
Epoch 7: val_accuracy did not improve from 0.49741
113/113 ————— 125s 1s/step - accuracy: 0.6090 - loss: 1.22
Epoch 8/10
113/113 ————— 0s 936ms/step - accuracy: 0.6352 - loss: 1.1
Epoch 8: val_accuracy did not improve from 0.49741
113/113 ————— 127s 1s/step - accuracy: 0.6301 - loss: 1.13
Epoch 9/10
113/113 ————— 0s 934ms/step - accuracy: 0.6298 - loss: 1.1
Epoch 9: ReduceLROnPlateau reducing learning rate to 9.000000427477062e-05.

Epoch 9: val_accuracy improved from 0.49741 to 0.51554, saving model to /content/plantdoc_results/saved_models/MobileNetV2_initial_best.keras
Epoch 9: finished saving model to /content/plantdoc_results/saved_models/MobileNetV2_initial_best.keras
113/113 ————— 127s 1s/step - accuracy: 0.6273 - loss: 1.14
Epoch 10/10
113/113 ————— 0s 941ms/step - accuracy: 0.6513 - loss: 1.0
Epoch 10: val_accuracy improved from 0.51554 to 0.51813, saving model to /content/plantdoc_results/saved_models/MobileNetV2_initial_best.keras
Epoch 10: finished saving model to /content/plantdoc_results/saved_models/MobileNetV2_initial_best.keras
113/113 ————— 143s 1s/step - accuracy: 0.6423 - loss: 1.08
Restoring model weights from the end of the best epoch: 10.

Fine-tuning selected layers...
Epoch 1/10
113/113 ————— 0s 1s/step - accuracy: 0.6484 - loss: 1.1200

```

```

113/113 ----- 0s 1s/step - accuracy: 0.6484 - loss: 1.1500
Epoch 1: val_accuracy improved from None to 0.48187, saving model to /content/plantdoc_results/save

Epoch 1: finished saving model to /content/plantdoc_results/saved_models/MobileNetV2_finetune_best.
113/113 ----- 165s 1s/step - accuracy: 0.6578 - loss: 1.08
Epoch 2/10
113/113 ----- 0s 1s/step - accuracy: 0.6628 - loss: 1.0383
Epoch 2: val_accuracy improved from 0.48187 to 0.51295, saving model to /content/plantdoc_results/sa

Epoch 2: finished saving model to /content/plantdoc_results/saved_models/MobileNetV2_finetune_best.
113/113 ----- 155s 1s/step - accuracy: 0.6622 - loss: 1.05
Epoch 3/10
113/113 ----- 0s 1s/step - accuracy: 0.6634 - loss: 1.0244
Epoch 3: val_accuracy improved from 0.51295 to 0.51813, saving model to /content/plantdoc_results/sa

Epoch 3: finished saving model to /content/plantdoc_results/saved_models/MobileNetV2_finetune_best.
113/113 ----- 210s 1s/step - accuracy: 0.6783 - loss: 0.98
Epoch 4/10
113/113 ----- 0s 1s/step - accuracy: 0.6999 - loss: 0.8788
Epoch 4: val_accuracy improved from 0.51813 to 0.52850, saving model to /content/plantdoc_results/sa

Epoch 4: finished saving model to /content/plantdoc_results/saved_models/MobileNetV2_finetune_best.
113/113 ----- 154s 1s/step - accuracy: 0.6911 - loss: 0.93
Epoch 5/10
113/113 ----- 0s 1s/step - accuracy: 0.6928 - loss: 0.8755
Epoch 5: val_accuracy improved from 0.52850 to 0.53109, saving model to /content/plantdoc_results/sa

Epoch 5: finished saving model to /content/plantdoc_results/saved_models/MobileNetV2_finetune_best.
113/113 ----- 155s 1s/step - accuracy: 0.6866 - loss: 0.92
Epoch 6/10
113/113 ----- 0s 1s/step - accuracy: 0.6966 - loss: 0.8798
Epoch 6: val_accuracy improved from 0.53109 to 0.53368, saving model to /content/plantdoc_results/sa

Epoch 6: finished saving model to /content/plantdoc_results/saved_models/MobileNetV2_finetune_best.
113/113 ----- 202s 1s/step - accuracy: 0.7022 - loss: 0.88
Epoch 7/10
113/113 ----- 0s 1s/step - accuracy: 0.6746 - loss: 0.9674
Epoch 7: val_accuracy did not improve from 0.53368
113/113 ----- 155s 1s/step - accuracy: 0.6916 - loss: 0.93
Epoch 8/10
113/113 ----- 0s 1s/step - accuracy: 0.7174 - loss: 0.8662
Epoch 8: val_accuracy did not improve from 0.53368
113/113 ----- 162s 1s/step - accuracy: 0.7027 - loss: 0.90
Epoch 9/10
113/113 ----- 0s 1s/step - accuracy: 0.7108 - loss: 0.8186
Epoch 9: ReduceLROnPlateau reducing learning rate to 2.9999999242136253e-06.

Epoch 9: val_accuracy did not improve from 0.53368
113/113 ----- 154s 1s/step - accuracy: 0.7060 - loss: 0.86
Epoch 10/10
113/113 ----- 0s 1s/step - accuracy: 0.7261 - loss: 0.8223
Epoch 10: val_accuracy improved from 0.53368 to 0.54145, saving model to /content/plantdoc_results/sa

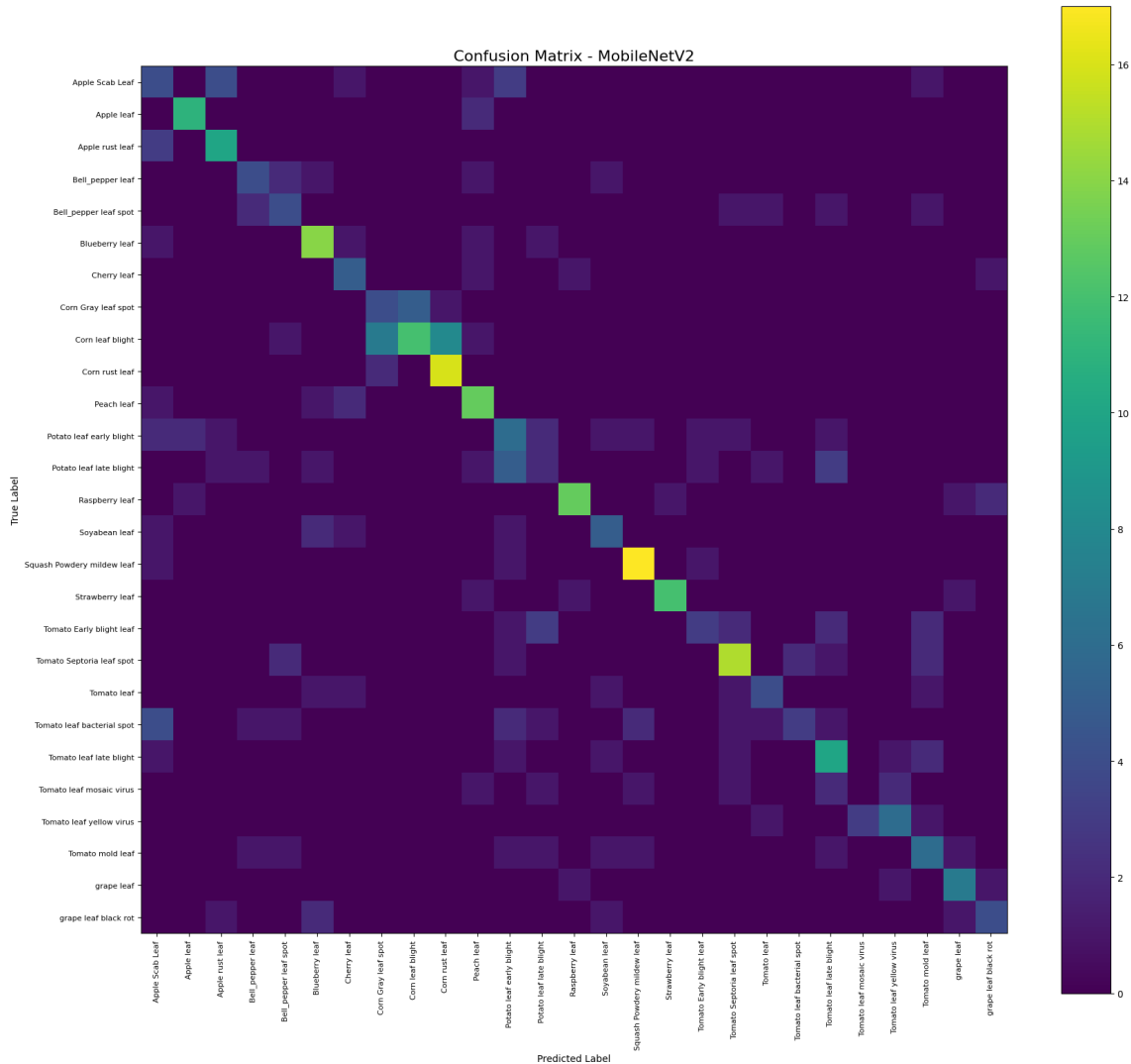
Epoch 10: finished saving model to /content/plantdoc_results/saved_models/MobileNetV2_finetune_best.
113/113 ----- 176s 2s/step - accuracy: 0.7199 - loss: 0.83
Restoring model weights from the end of the best epoch: 10.
25/25 ----- 22s 785ms/step

```

Evaluation Summary:

Accuracy: 0.5426356589147286

Accuracy: 0.572033039177280
 Precision Macro: 0.5166456245122059
 Recall Macro: 0.5243269793573074
 F1 Macro: 0.5087199783174854
 Model Size MB: 26.644204139709473
 Training Time min: 50.3247987985611
 Inference Time/Image sec: 0.05806765815084295



Saved: /content/plantdoc_results/figures/MobileNetV2_confusion_matrix.png

Training Model: ResNet50

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50_weights_tf_dim_ordering_tf_data_format.h5 94765736/94765736 — 0s 0us/step

Initial training with frozen base model...

Epoch 1/10

113/113 — 0s 3s/step - accuracy: 0.1524 - loss: 3.9444

Epoch 1: val_accuracy improved from None to 0.37565, saving model to /content/plantdoc_results/saved_models/ResNet50_initial_best.keras

Epoch 1: finished saving model to /content/plantdoc_results/saved_models/ResNet50_initial_best.keras

113/113 — 440s 4s/step - accuracy: 0.2207 - loss: 3.41

Epoch 2/10

113/113 — 0s 3s/step - accuracy: 0.4026 - loss: 2.2166

Epoch 2: val_accuracy improved from 0.37565 to 0.50000, saving model to /content/plantdoc_results/saved_models/ResNet50_initial_best.keras

Epoch 2: finished saving model to /content/plantdoc_results/saved_models/ResNet50_initial_best.keras

113/113 — 435s 4s/step - accuracy: 0.3932 - loss: 2.24

Epoch 3/10

```
Epoch 3/10
113/113 ————— 0s 3s/step - accuracy: 0.4654 - loss: 1.8948
Epoch 3: val_accuracy improved from 0.50000 to 0.50518, saving model to /content/plantdoc_results/sa

Epoch 3: finished saving model to /content/plantdoc_results/saved_models/ResNet50_initial_best.keras
113/113 ————— 442s 4s/step - accuracy: 0.4714 - loss: 1.94
Epoch 4/10
113/113 ————— 0s 3s/step - accuracy: 0.5548 - loss: 1.5818
Epoch 4: val_accuracy did not improve from 0.50518
113/113 ————— 451s 4s/step - accuracy: 0.5336 - loss: 1.62
Epoch 5/10
113/113 ————— 0s 3s/step - accuracy: 0.5761 - loss: 1.3923
Epoch 5: ReduceLROnPlateau reducing learning rate to 0.0003000000142492354.

Epoch 5: val_accuracy improved from 0.50518 to 0.51295, saving model to /content/plantdoc_results/sa

Epoch 5: finished saving model to /content/plantdoc_results/saved_models/ResNet50_initial_best.keras
113/113 ————— 435s 4s/step - accuracy: 0.5779 - loss: 1.43
Epoch 6/10
113/113 ————— 0s 3s/step - accuracy: 0.6174 - loss: 1.1591
Epoch 6: val_accuracy improved from 0.51295 to 0.51813, saving model to /content/plantdoc_results/sa

Epoch 6: finished saving model to /content/plantdoc_results/saved_models/ResNet50_initial_best.keras
113/113 ————— 425s 4s/step - accuracy: 0.6278 - loss: 1.15
Epoch 7/10
113/113 ————— 0s 3s/step - accuracy: 0.6536 - loss: 1.1024
Epoch 7: val_accuracy improved from 0.51813 to 0.53109, saving model to /content/plantdoc_results/sa

Epoch 7: finished saving model to /content/plantdoc_results/saved_models/ResNet50_initial_best.keras
113/113 ————— 424s 4s/step - accuracy: 0.6467 - loss: 1.09
Epoch 8/10
113/113 ————— 0s 3s/step - accuracy: 0.6544 - loss: 1.0804
Epoch 8: val_accuracy improved from 0.53109 to 0.53368, saving model to /content/plantdoc_results/sa

Epoch 8: finished saving model to /content/plantdoc_results/saved_models/ResNet50_initial_best.keras
113/113 ————— 423s 4s/step - accuracy: 0.6556 - loss: 1.09
Epoch 9/10
113/113 ————— 0s 3s/step - accuracy: 0.6787 - loss: 0.9740
Epoch 9: val_accuracy improved from 0.53368 to 0.54404, saving model to /content/plantdoc_results/sa

Epoch 9: finished saving model to /content/plantdoc_results/saved_models/ResNet50_initial_best.keras
113/113 ————— 434s 4s/step - accuracy: 0.6816 - loss: 0.97
Epoch 10/10
113/113 ————— 0s 3s/step - accuracy: 0.6821 - loss: 0.9566
Epoch 10: val_accuracy did not improve from 0.54404
113/113 ————— 441s 4s/step - accuracy: 0.6905 - loss: 0.96
Restoring model weights from the end of the best epoch: 9.

Fine-tuning selected layers...
Epoch 1/10
113/113 ————— 0s 6s/step - accuracy: 0.6928 - loss: 0.9120
Epoch 1: val_accuracy improved from None to 0.55181, saving model to /content/plantdoc_results/save

Epoch 1: finished saving model to /content/plantdoc_results/saved_models/ResNet50_finetune_best.keras
113/113 ————— 725s 6s/step - accuracy: 0.6889 - loss: 0.93
Epoch 2/10
113/113 ————— 0s 6s/step - accuracy: 0.6925 - loss: 0.9030
Epoch 2: val_accuracy did not improve from 0.55181
113/113 ————— 703s 6s/step - accuracy: 0.6994 - loss: 0.87
Epoch 3/10
```

113/113 ————— 0s 5s/step - accuracy: 0.7348 - loss: 0.7783
 Epoch 3: val_accuracy improved from 0.55181 to 0.55440, saving model to /content/plantdoc_results/saved_models/ResNet50_finetune_best.keras

Epoch 3: finished saving model to /content/plantdoc_results/saved_models/ResNet50_finetune_best.keras
 113/113 ————— 712s 6s/step - accuracy: 0.7316 - loss: 0.7811
 Epoch 4/10

113/113 ————— 0s 5s/step - accuracy: 0.7726 - loss: 0.7140
 Epoch 4: val_accuracy improved from 0.55440 to 0.56736, saving model to /content/plantdoc_results/saved_models/ResNet50_finetune_best.keras

Epoch 4: finished saving model to /content/plantdoc_results/saved_models/ResNet50_finetune_best.keras
 113/113 ————— 698s 6s/step - accuracy: 0.7471 - loss: 0.7411
 Epoch 5/10

113/113 ————— 0s 5s/step - accuracy: 0.7847 - loss: 0.6374
 Epoch 5: ReduceLROnPlateau reducing learning rate to 2.9999999242136253e-06.

Epoch 5: val_accuracy improved from 0.56736 to 0.58808, saving model to /content/plantdoc_results/saved_models/ResNet50_finetune_best.keras

Epoch 5: finished saving model to /content/plantdoc_results/saved_models/ResNet50_finetune_best.keras
 113/113 ————— 711s 6s/step - accuracy: 0.7676 - loss: 0.6811
 Epoch 6/10

113/113 ————— 0s 5s/step - accuracy: 0.7843 - loss: 0.5933
 Epoch 6: val_accuracy did not improve from 0.58808

113/113 ————— 691s 6s/step - accuracy: 0.7687 - loss: 0.6211
 Epoch 7/10

113/113 ————— 0s 5s/step - accuracy: 0.7537 - loss: 0.7140
 Epoch 7: val_accuracy did not improve from 0.58808

113/113 ————— 696s 6s/step - accuracy: 0.7698 - loss: 0.6611
 Epoch 8/10

113/113 ————— 0s 5s/step - accuracy: 0.8040 - loss: 0.6164
 Epoch 8: val_accuracy did not improve from 0.58808

113/113 ————— 691s 6s/step - accuracy: 0.7903 - loss: 0.6011
 Epoch 9/10

113/113 ————— 0s 5s/step - accuracy: 0.8038 - loss: 0.5667
 Epoch 9: val_accuracy did not improve from 0.58808

113/113 ————— 694s 6s/step - accuracy: 0.7870 - loss: 0.6111
 Epoch 10/10

113/113 ————— 0s 5s/step - accuracy: 0.7981 - loss: 0.5723
 Epoch 10: ReduceLROnPlateau reducing learning rate to 8.999999636216671e-07.

Epoch 10: val_accuracy did not improve from 0.58808

113/113 ————— 747s 6s/step - accuracy: 0.7876 - loss: 0.5911
 Epoch 10: early stopping
 Restoring model weights from the end of the best epoch: 5.

25/25 ————— 77s 3s/step

Evaluation Summary:

Accuracy: 0.5529715762273901

Precision Macro: 0.5528640930366526

Recall Macro: 0.5425078602835313

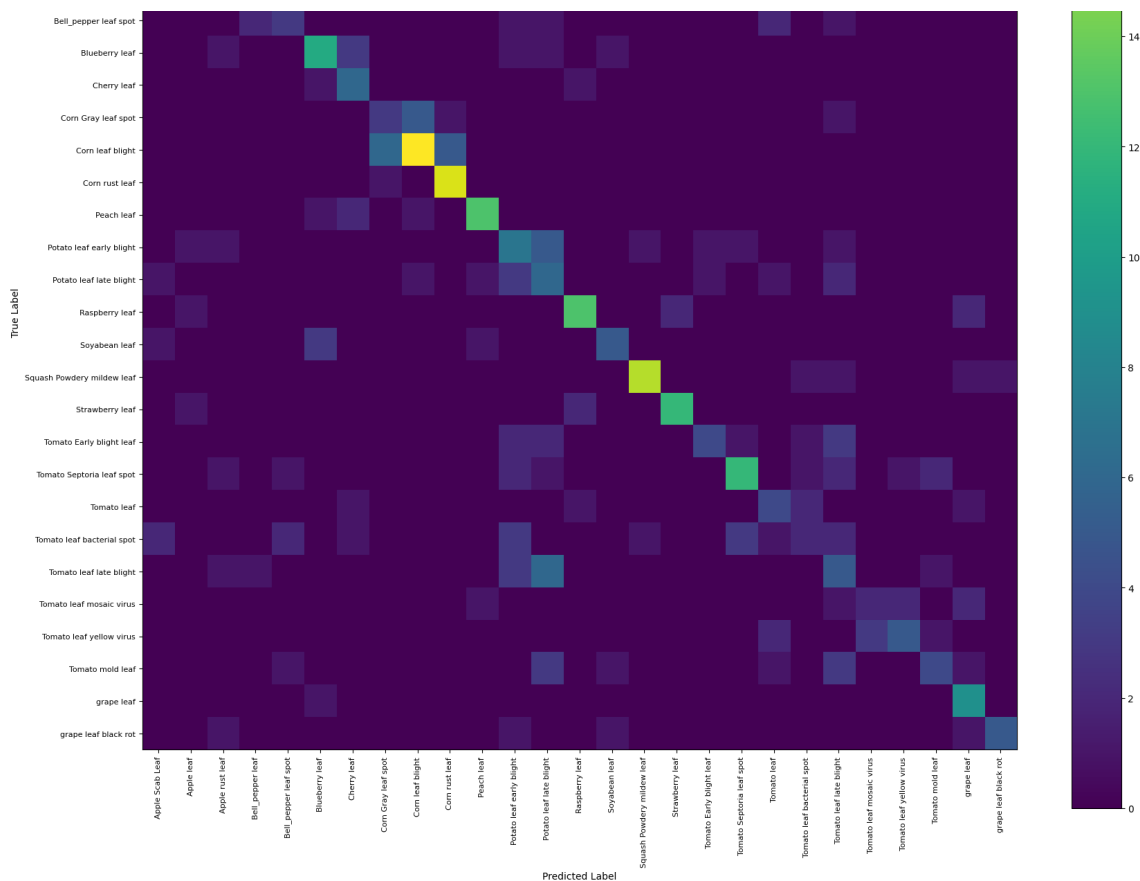
F1 Macro: 0.5331596132600323

Model Size MB: 227.8972568511963

Training Time min: 190.48109938700995

Inference Time/Image sec: 0.19837502422874903





Saved: /content/plantdoc_results/figures/ResNet50_confusion_matrix.png

Training Model: EfficientNetB0

Downloading data from https://storage.googleapis.com/keras-applications/efficientnetb0_notop.h5
16705208/16705208 ————— **0s** 0us/step

Initial training with frozen base model...

Epoch 1/10

113/113 ————— **0s** 1s/step - accuracy: 0.1231 - loss: 3.8698

Epoch 1: val_accuracy improved from None to 0.44301, saving model to /content/plantdoc_results/save

Epoch 1: finished saving model to /content/plantdoc_results/saved_models/EfficientNetB0_initial_best.k

113/113 ————— **213s** 2s/step - accuracy: 0.1925 - loss: 3.33

Epoch 2/10

113/113 ————— **0s** 1s/step - accuracy: 0.3653 - loss: 2.3061

Epoch 2: val_accuracy improved from 0.44301 to 0.46891, saving model to /content/plantdoc_results/sa

Epoch 2: finished saving model to /content/plantdoc_results/saved_models/EfficientNetB0_initial_best.k

113/113 ————— **196s** 2s/step - accuracy: 0.3716 - loss: 2.26

Epoch 3/10

113/113 ————— **0s** 1s/step - accuracy: 0.4691 - loss: 1.8524

Epoch 3: val_accuracy improved from 0.46891 to 0.49223, saving model to /content/plantdoc_results/sa

Epoch 3: finished saving model to /content/plantdoc_results/saved_models/EfficientNetB0_initial_best.k

113/113 ————— **214s** 2s/step - accuracy: 0.4570 - loss: 1.89

Epoch 4/10

113/113 ————— **0s** 1s/step - accuracy: 0.5045 - loss: 1.6345

Epoch 4: val_accuracy improved from 0.49223 to 0.49741, saving model to /content/plantdoc_results/sa

Epoch 4: finished saving model to /content/plantdoc_results/saved_models/EfficientNetB0_initial_best.k

113/113 ————— **192s** 2s/step - accuracy: 0.5130 - loss: 1.58

Epoch 5/10

```

113/113 _____ 0s 1s/step - accuracy: 0.5238 - loss: 1.5366
Epoch 5: val_accuracy did not improve from 0.49741
113/113 _____ 182s 2s/step - accuracy: 0.5202 - loss: 1.53
Epoch 6/10
113/113 _____ 0s 1s/step - accuracy: 0.5854 - loss: 1.2920
Epoch 6: ReduceLROnPlateau reducing learning rate to 0.0003000000142492354.

Epoch 6: val_accuracy improved from 0.49741 to 0.52332, saving model to /content/plantdoc_results/sa

Epoch 6: finished saving model to /content/plantdoc_results/saved_models/EfficientNetB0_initial_best.k
113/113 _____ 194s 2s/step - accuracy: 0.5740 - loss: 1.34
Epoch 7/10
113/113 _____ 0s 1s/step - accuracy: 0.6103 - loss: 1.1273
Epoch 7: val_accuracy did not improve from 0.52332
113/113 _____ 183s 2s/step - accuracy: 0.6190 - loss: 1.15
Epoch 8/10
113/113 _____ 0s 1s/step - accuracy: 0.6453 - loss: 1.0992
Epoch 8: val_accuracy did not improve from 0.52332
113/113 _____ 194s 2s/step - accuracy: 0.6384 - loss: 1.13
Epoch 9/10
113/113 _____ 0s 1s/step - accuracy: 0.6391 - loss: 1.1108
Epoch 9: ReduceLROnPlateau reducing learning rate to 9.000000427477062e-05.

Epoch 9: val_accuracy did not improve from 0.52332
113/113 _____ 195s 2s/step - accuracy: 0.6428 - loss: 1.07
Epoch 10/10
113/113 _____ 0s 1s/step - accuracy: 0.6676 - loss: 1.0330
Epoch 10: val_accuracy did not improve from 0.52332
113/113 _____ 190s 2s/step - accuracy: 0.6689 - loss: 1.03
Restoring model weights from the end of the best epoch: 6.

Fine-tuning selected layers...
Epoch 1/10
113/113 _____ 0s 2s/step - accuracy: 0.5953 - loss: 1.2117
Epoch 1: val_accuracy improved from None to 0.54145, saving model to /content/plantdoc_results/save

Epoch 1: finished saving model to /content/plantdoc_results/saved_models/EfficientNetB0_finetune_bes
113/113 _____ 254s 2s/step - accuracy: 0.6273 - loss: 1.13
Epoch 2/10
113/113 _____ 0s 2s/step - accuracy: 0.6364 - loss: 1.0922
Epoch 2: val_accuracy did not improve from 0.54145
113/113 _____ 230s 2s/step - accuracy: 0.6267 - loss: 1.14
Epoch 3/10
113/113 _____ 0s 2s/step - accuracy: 0.6313 - loss: 1.1429
Epoch 3: val_accuracy did not improve from 0.54145
113/113 _____ 224s 2s/step - accuracy: 0.6461 - loss: 1.06
Epoch 4/10
113/113 _____ 0s 2s/step - accuracy: 0.6603 - loss: 1.0274
Epoch 4: val_accuracy did not improve from 0.54145
113/113 _____ 238s 2s/step - accuracy: 0.6650 - loss: 1.00
Epoch 5/10
113/113 _____ 0s 2s/step - accuracy: 0.6854 - loss: 0.9663
Epoch 5: val_accuracy improved from 0.54145 to 0.54404, saving model to /content/plantdoc_results/sa

Epoch 5: finished saving model to /content/plantdoc_results/saved_models/EfficientNetB0_finetune_bes
113/113 _____ 238s 2s/step - accuracy: 0.6711 - loss: 0.97
Epoch 6/10
113/113 _____ 0s 2s/step - accuracy: 0.6746 - loss: 1.0210
Epoch 6: val_accuracy improved from 0.54404 to 0.54663, saving model to /content/plantdoc_results/sa

```

Epoch 6: finished saving model to /content/plantdoc_results/saved_models/EfficientNetB0_finetune_bes
113/113 ————— **242s** 2s/step - accuracy: 0.6617 - loss: 1.03
Epoch 7/10
113/113 ————— **0s** 2s/step - accuracy: 0.6564 - loss: 1.0013
Epoch 7: val_accuracy did not improve from 0.54663
113/113 ————— **239s** 2s/step - accuracy: 0.6694 - loss: 0.97
Epoch 8/10
113/113 ————— **0s** 2s/step - accuracy: 0.6931 - loss: 0.9445
Epoch 8: val_accuracy did not improve from 0.54663
113/113 ————— **227s** 2s/step - accuracy: 0.6778 - loss: 0.95
Epoch 9/10
113/113 ————— **0s** 2s/step - accuracy: 0.7067 - loss: 0.8770
Epoch 9: val_accuracy did not improve from 0.54663
113/113 ————— **229s** 2s/step - accuracy: 0.6988 - loss: 0.89
Epoch 10/10
113/113 ————— **0s** 2s/step - accuracy: 0.6990 - loss: 0.8886
Epoch 10: val_accuracy did not improve from 0.54663
113/113 ————— **239s** 2s/step - accuracy: 0.6944 - loss: 0.90
Restoring model weights from the end of the best epoch: 6.
25/25 ————— **34s** 1s/step

Evaluation Summary:

Accuracy: 0.5633074935400517

Precision Macro: 0.5547879317487161

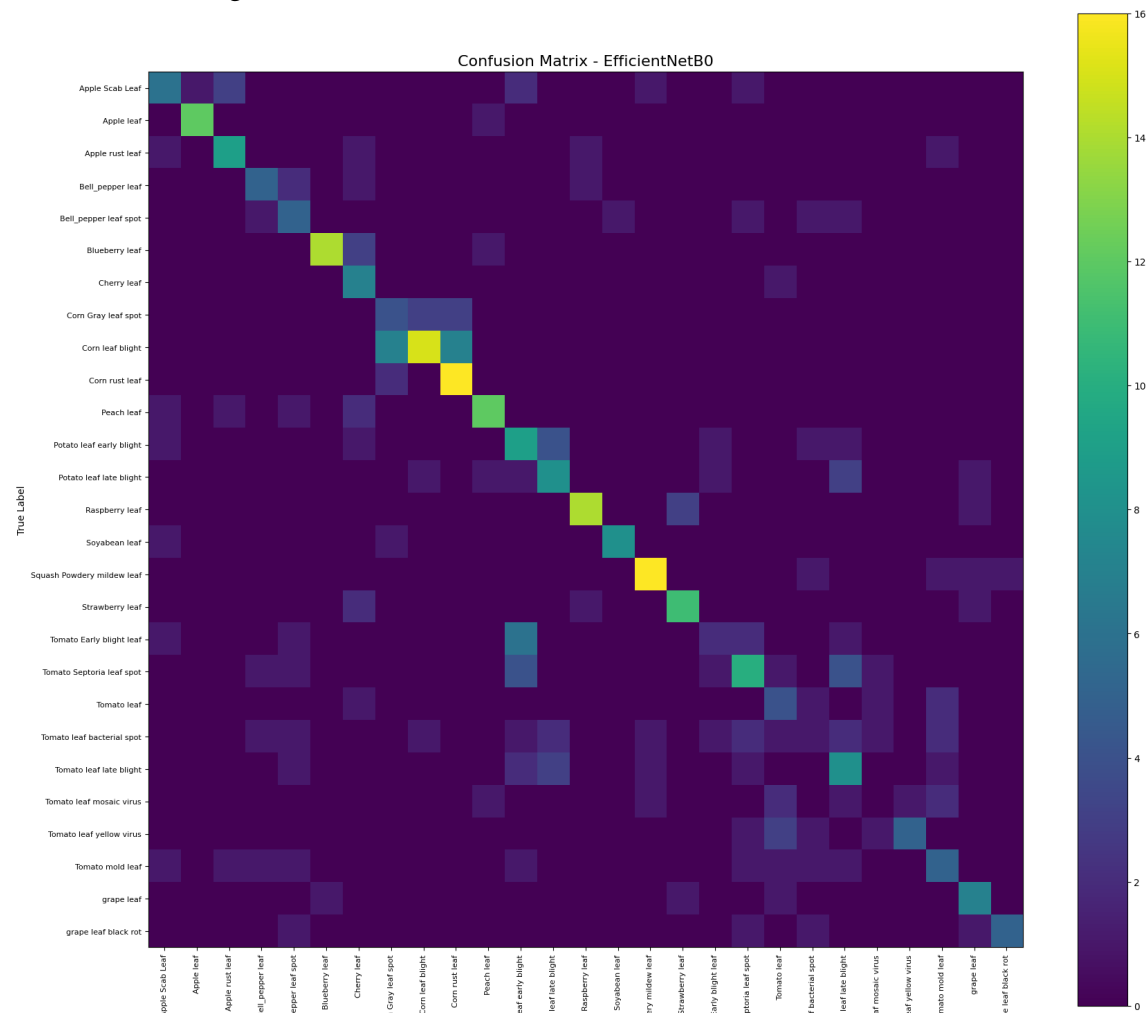
Recall Macro: 0.555745998091737

F1 Macro: 0.5422994771511939

Model Size MB: 43.46766471862793

Training Time min: 71.8997054696083

Inference Time/Image sec: 0.0888041926292794



Saved: /content/plantdoc_results/figures/EfficientNetB0_confusion_matrix.png

Training Model: DenseNet121

Downloading data from <https://storage.googleapis.com/tensorflow/keras-applications/densenet/densenet29084464/29084464> ————— 0s 0us/step

Initial training with frozen base model...

Epoch 1/10

113/113 ————— 0s 3s/step - accuracy: 0.1263 - loss: 3.4061

Epoch 1: val_accuracy improved from None to 0.37565, saving model to /content/plantdoc_results/saved_models/DenseNet121_initial_best.keras

Epoch 1: finished saving model to /content/plantdoc_results/saved_models/DenseNet121_initial_best.keras

113/113 ————— 423s 4s/step - accuracy: 0.2058 - loss: 2.9705

Epoch 2/10

113/113 ————— 0s 3s/step - accuracy: 0.3789 - loss: 2.0327

Epoch 2: val_accuracy improved from 0.37565 to 0.47150, saving model to /content/plantdoc_results/saved_models/DenseNet121_initial_best.keras

Epoch 2: finished saving model to /content/plantdoc_results/saved_models/DenseNet121_initial_best.keras

113/113 ————— 411s 4s/step - accuracy: 0.3882 - loss: 2.0105

Epoch 3/10

113/113 ————— 0s 3s/step - accuracy: 0.4477 - loss: 1.8386

Epoch 3: val_accuracy improved from 0.47150 to 0.48964, saving model to /content/plantdoc_results/saved_models/DenseNet121_initial_best.keras

Epoch 3: finished saving model to /content/plantdoc_results/saved_models/DenseNet121_initial_best.keras

113/113 ————— 411s 4s/step - accuracy: 0.4615 - loss: 1.7905

Epoch 4/10

113/113 ————— 0s 3s/step - accuracy: 0.5295 - loss: 1.5870

Epoch 4: val_accuracy improved from 0.48964 to 0.52332, saving model to /content/plantdoc_results/saved_models/DenseNet121_initial_best.keras

Epoch 4: finished saving model to /content/plantdoc_results/saved_models/DenseNet121_initial_best.keras

113/113 ————— 398s 4s/step - accuracy: 0.5208 - loss: 1.5505

Epoch 5/10

113/113 ————— 0s 3s/step - accuracy: 0.5234 - loss: 1.5030

Epoch 5: val_accuracy improved from 0.52332 to 0.53109, saving model to /content/plantdoc_results/saved_models/DenseNet121_initial_best.keras

Epoch 5: finished saving model to /content/plantdoc_results/saved_models/DenseNet121_initial_best.keras

113/113 ————— 405s 4s/step - accuracy: 0.5330 - loss: 1.4405

Epoch 6/10

113/113 ————— 0s 3s/step - accuracy: 0.5920 - loss: 1.3589

Epoch 6: val_accuracy did not improve from 0.53109

113/113 ————— 404s 4s/step - accuracy: 0.5657 - loss: 1.4105

Epoch 7/10

113/113 ————— 0s 3s/step - accuracy: 0.5966 - loss: 1.2335

Epoch 7: val_accuracy improved from 0.53109 to 0.54922, saving model to /content/plantdoc_results/saved_models/DenseNet121_initial_best.keras

Epoch 7: finished saving model to /content/plantdoc_results/saved_models/DenseNet121_initial_best.keras

113/113 ————— 418s 4s/step - accuracy: 0.5807 - loss: 1.2605

Epoch 8/10

113/113 ————— 0s 3s/step - accuracy: 0.6011 - loss: 1.2020

Epoch 8: val_accuracy did not improve from 0.54922

113/113 ————— 440s 4s/step - accuracy: 0.5935 - loss: 1.2205

Epoch 9/10

113/113 ————— 0s 3s/step - accuracy: 0.6260 - loss: 1.1111

Epoch 9: ReduceLROnPlateau reducing learning rate to 0.0003000000142492354.

Epoch 9: val_accuracy did not improve from 0.54922

113/113 ————— **414s** 4s/step - accuracy: 0.6206 - loss: 1.13

Epoch 10/10

113/113 ————— **0s** 3s/step - accuracy: 0.6415 - loss: 1.0735

Epoch 10: val_accuracy improved from 0.54922 to 0.55440, saving model to /content/plantdoc_results/saved_models/DenseNet121_initial_best.k

Epoch 10: finished saving model to /content/plantdoc_results/saved_models/DenseNet121_initial_best.k

113/113 ————— **417s** 4s/step - accuracy: 0.6339 - loss: 1.08

Restoring model weights from the end of the best epoch: 10.

Fine-tuning selected layers...

Epoch 1/10

113/113 ————— **0s** 3s/step - accuracy: 0.6793 - loss: 0.9733

Epoch 1: val_accuracy improved from None to 0.54663, saving model to /content/plantdoc_results/saved_models/DenseNet121_finetune_best.k

Epoch 1: finished saving model to /content/plantdoc_results/saved_models/DenseNet121_finetune_best.k

113/113 ————— **504s** 4s/step - accuracy: 0.6800 - loss: 0.97

Epoch 2/10

113/113 ————— **0s** 3s/step - accuracy: 0.6870 - loss: 0.9655

Epoch 2: val_accuracy did not improve from 0.54663

113/113 ————— **457s** 4s/step - accuracy: 0.6900 - loss: 0.95

Epoch 3/10

113/113 ————— **0s** 3s/step - accuracy: 0.6779 - loss: 0.9808

Epoch 3: val_accuracy improved from 0.54663 to 0.55440, saving model to /content/plantdoc_results/saved_models/DenseNet121_finetune_best.k

Epoch 3: finished saving model to /content/plantdoc_results/saved_models/DenseNet121_finetune_best.k

113/113 ————— **475s** 4s/step - accuracy: 0.6761 - loss: 0.96

Epoch 4/10

113/113 ————— **0s** 3s/step - accuracy: 0.7033 - loss: 0.8571

Epoch 4: val_accuracy improved from 0.55440 to 0.55959, saving model to /content/plantdoc_results/saved_models/DenseNet121_finetune_best.k

Epoch 4: finished saving model to /content/plantdoc_results/saved_models/DenseNet121_finetune_best.k

113/113 ————— **457s** 4s/step - accuracy: 0.7110 - loss: 0.88

Epoch 5/10

113/113 ————— **0s** 3s/step - accuracy: 0.7132 - loss: 0.8942

Epoch 5: val_accuracy improved from 0.55959 to 0.56218, saving model to /content/plantdoc_results/saved_models/DenseNet121_finetune_best.k

Epoch 5: finished saving model to /content/plantdoc_results/saved_models/DenseNet121_finetune_best.k

113/113 ————— **471s** 4s/step - accuracy: 0.7166 - loss: 0.88

Epoch 6/10

113/113 ————— **0s** 3s/step - accuracy: 0.6903 - loss: 0.8912

Epoch 6: val_accuracy improved from 0.56218 to 0.56477, saving model to /content/plantdoc_results/saved_models/DenseNet121_finetune_best.k

Epoch 6: finished saving model to /content/plantdoc_results/saved_models/DenseNet121_finetune_best.k

113/113 ————— **472s** 4s/step - accuracy: 0.6850 - loss: 0.93

Epoch 7/10

113/113 ————— **0s** 3s/step - accuracy: 0.7036 - loss: 0.8660

Epoch 7: val_accuracy did not improve from 0.56477

113/113 ————— **457s** 4s/step - accuracy: 0.7138 - loss: 0.84

Epoch 8/10

113/113 ————— **0s** 3s/step - accuracy: 0.7343 - loss: 0.8403

Epoch 8: val_accuracy did not improve from 0.56477

113/113 ————— **453s** 4s/step - accuracy: 0.7083 - loss: 0.86

Epoch 9/10

113/113 ————— **0s** 3s/step - accuracy: 0.7117 - loss: 0.8382

Epoch 9: val_accuracy did not improve from 0.56477

113/113 ————— **521s** 4s/step - accuracy: 0.7116 - loss: 0.86

Epoch 10/10

113/113 ————— **0s** 3s/step - accuracy: 0.7372 - loss: 0.7614

Epoch 10: ReduceLROnPlateau reducing learning rate to 2.9999999242136253e-06.

Epoch 10: val_accuracy improved from 0.56477 to 0.57254, saving model to /content/plantdoc_results/

Epoch 10: finished saving model to /content/plantdoc_results/saved_models/DenseNet121_finetune_bes
113/113

488s 4s/step - accuracy: 0.7138 - loss: 0.82

Restoring model weights from the end of the best epoch: 10.

25/25 **76s** 3s/step

Evaluation Summary:

Accuracy: 0.6020671834625323

Precision Macro: 0.6021495204264931

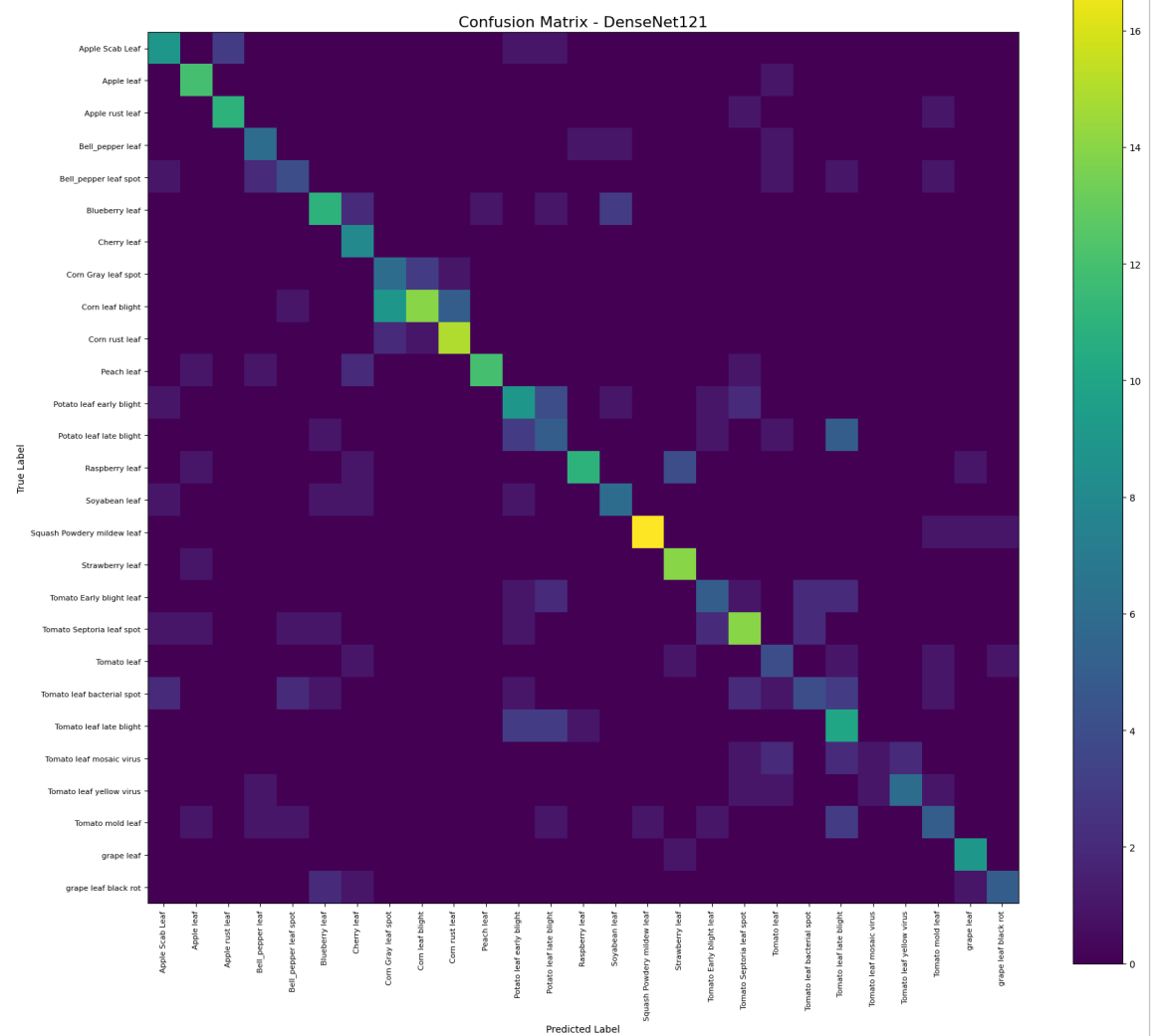
Recall Macro: 0.6023415663861372

F1 Macro: 0.5847225666553142

Model Size MB: 53.22610378265381

Training Time min: 148.30860479275387

Inference Time/Image sec: 0.19742950358132058



Saved: /content/plantdoc_results/figures/DenseNet121_confusion_matrix.png

All models trained and evaluated.



Start coding or generate with AI.

```

# =====
# STEP 17: CREATE FINAL RESULT TABLE
# =====

summary_rows = []

for result in all_results:
    summary_rows.append({
        "Model": result["Model"],
        "Accuracy": result["Accuracy"],
        "Precision_Macro": result["Precision_Macro"],
        "Recall_Macro": result["Recall_Macro"],
        "F1_Macro": result["F1_Macro"],
        "Model_Size_MB": result["Model_Size_MB"],
        "Total_Parameters": result["Total_Parameters"],
        "Training_Time_min": result["Training_Time_min"],
        "Inference_Time_per_Image_sec": result["Inference_Time_per_
    })

results_df = pd.DataFrame(summary_rows)

# Sort by F1-score first
results_df = results_df.sort_values(by="F1_Macro", ascending=False)

display(results_df)

csv_path = RESULT_DIR / "five_model_comparative_analysis_results.csv"
results_df.to_csv(csv_path, index=False)

print("Saved CSV:", csv_path)

```

	Model	Accuracy	Precision_Macro	Recall_Macro	F1_Macro	Model_Size_MB
0	DenseNet121	0.602067	0.602150	0.602342	0.584723	53.22610
1	EfficientNetB0	0.563307	0.554788	0.555746	0.542299	43.46766
2	ResNet50	0.552972	0.552864	0.542508	0.533160	227.89725
3	MobileNetV2	0.542636	0.516646	0.524327	0.508720	26.64420
4	CustomCNN	0.211886	0.232701	0.193176	0.186717	5.36727

Saved CSV: /content/plantdoc_results/five_model_comparative_analysis_results.csv

Next steps:

[Generate code with results_df](#)

[New interactive sheet](#)

```

# =====
# STEP 18: CREATE RESULT TABLE
# =====

paper_table = results_df.copy()

paper_table["Accuracy (%)"] = paper_table["Accuracy"] * 100
paper_table["Precision (%)"] = paper_table["Precision_Macro"] * 100
paper_table["Recall (%)"] = paper_table["Recall_Macro"] * 100
paper_table["F1-score (%)"] = paper_table["F1_Macro"] * 100

paper_table = paper_table[[
    "Model",
    "Accuracy (%)",
    "Precision (%)",
    "Recall (%)",
    "F1-score (%)",
    "Model_Size_MB",
    "Training_Time_min",
    "Inference_Time_per_Image_sec",
    "Total_Parameters"
]]

paper_table = paper_table.round(4)

display(paper_table)

paper_csv_path = RESULT_DIR / "ieee_paper_result_table.csv"
paper_table.to_csv(paper_csv_path, index=False)

print("Saved paper table:", paper_csv_path)

```

	Model	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)	Model_Size_MB	Training_Time_min
0	DenseNet121	60.2067	60.2150	60.2342	58.4723	53.2261	14
1	EfficientNetB0	56.3307	55.4788	55.5746	54.2299	43.4677	7
2	ResNet50	55.2972	55.2864	54.2508	53.3160	227.8973	19
3	MobileNetV2	54.2636	51.6646	52.4327	50.8720	26.6442	5
4	CustomCNN	21.1886	23.2701	19.3176	18.6717	5.3673	14

----- Saved IEEE paper table: /content/nlantic_results/ieee_paper_result_table.csv -----

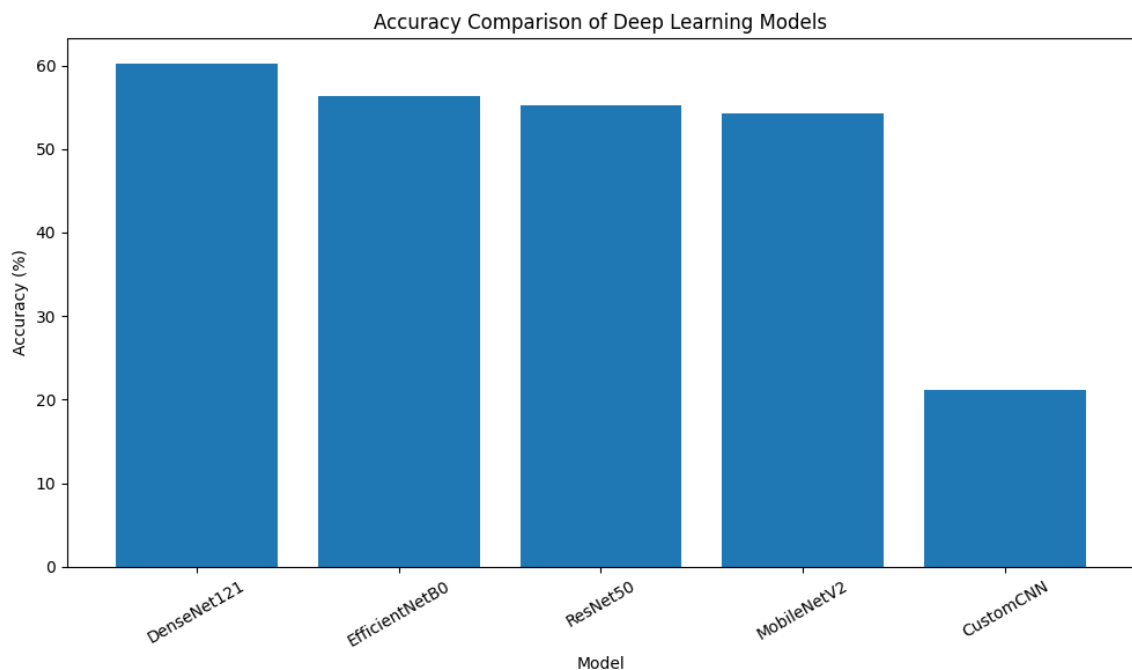
Next steps:

[Generate code with paper_table](#)

[New interactive sheet](#)

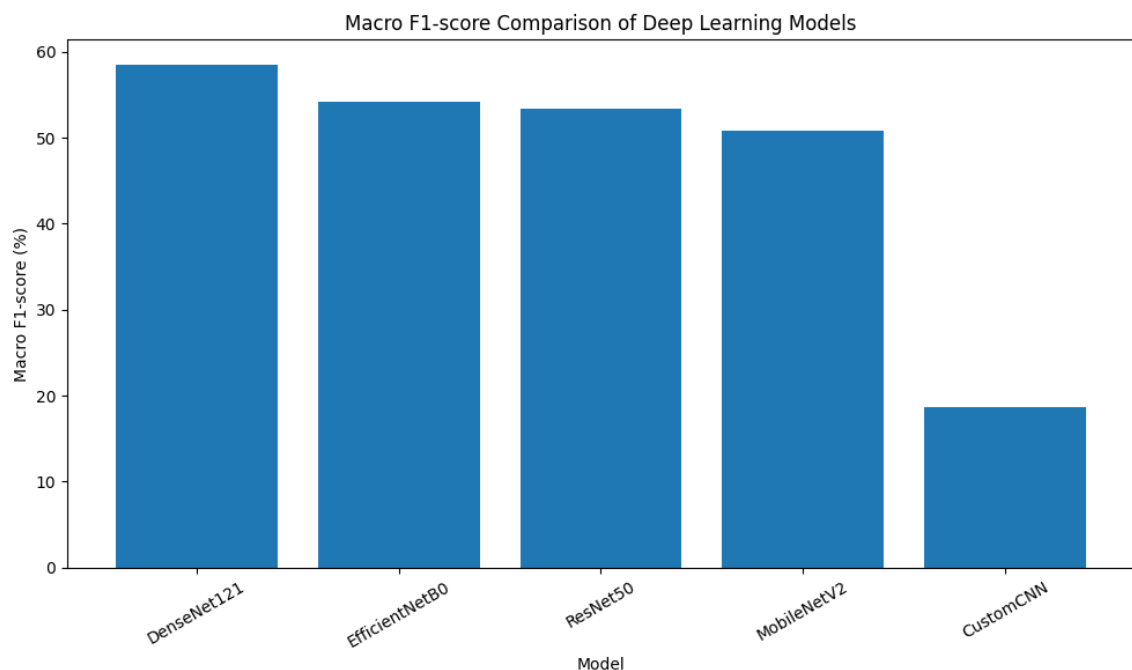
Start coding or [generate](#) with AI.

```
# =====  
# STEP 19: ACCURACY COMPARISON GRAPH  
# =====  
  
plt.figure(figsize=(10, 6))  
plt.bar(results_df["Model"], results_df["Accuracy"] * 100)  
plt.xlabel("Model")  
plt.ylabel("Accuracy (%)")  
plt.title("Accuracy Comparison of Deep Learning Models")  
plt.xticks(rotation=30)  
plt.tight_layout()  
  
save_path = FIGURE_DIR / "accuracy_comparison.png"  
plt.savefig(save_path, dpi=300, bbox_inches="tight")  
plt.show()  
  
print("Saved:", save_path)
```



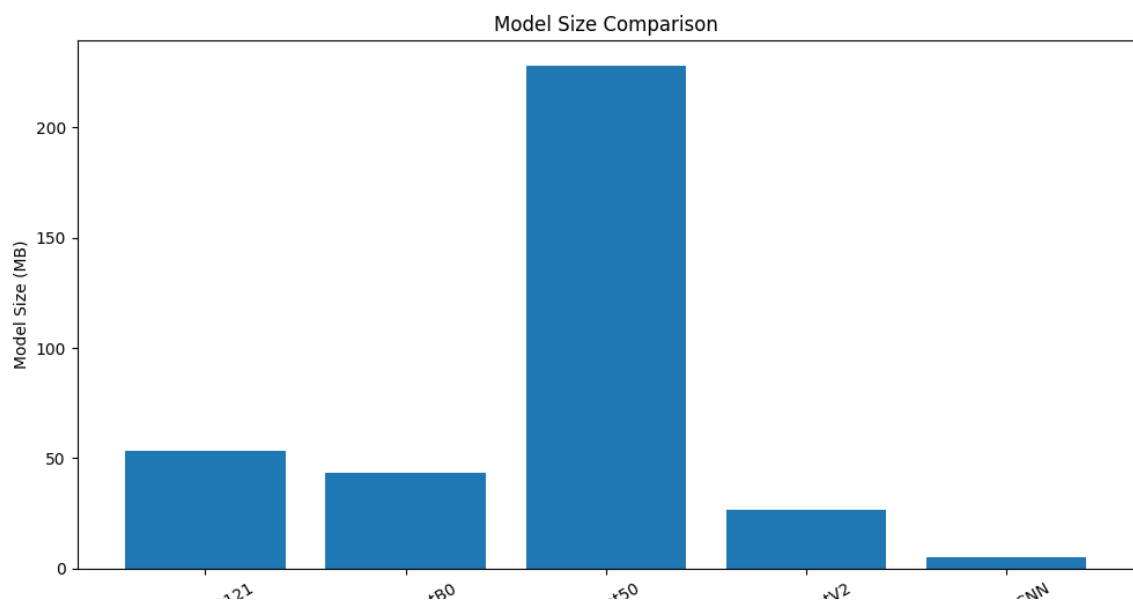
Saved: /content/plantdoc_results/figures/accuracy_comparison.png

```
# =====  
# STEP 20: F1-SCORE COMPARISON GRAPH  
# =====  
  
plt.figure(figsize=(10, 6))  
plt.bar(results_df["Model"], results_df["F1_Macro"] * 100)  
plt.xlabel("Model")  
plt.ylabel("Macro F1-score (%)")  
plt.title("Macro F1-score Comparison of Deep Learning Models")  
plt.xticks(rotation=30)  
plt.tight_layout()  
  
save_path = FIGURE_DIR / "f1_score_comparison.png"  
plt.savefig(save_path, dpi=300, bbox_inches="tight")  
plt.show()  
  
print("Saved:", save_path)
```



Saved: /content/plantdoc_results/figures/f1_score_comparison.png

```
# =====  
# STEP 21: MODEL SIZE COMPARISON GRAPH  
# =====  
  
plt.figure(figsize=(10, 6))  
plt.bar(results_df["Model"], results_df["Model_Size_MB"])  
plt.xlabel("Model")  
plt.ylabel("Model Size (MB)")  
plt.title("Model Size Comparison")  
plt.xticks(rotation=30)  
plt.tight_layout()  
  
save_path = FIGURE_DIR / "model_size_comparison.png"  
plt.savefig(save_path, dpi=300, bbox_inches="tight")  
plt.show()  
  
print("Saved:", save_path)
```



DenseNet...
 EfficientNet...
 ResNe...
 MobileNe...
 Custom...
 Model

Saved: /content/plantdoc_results/figures/model_size_comparison.png

```

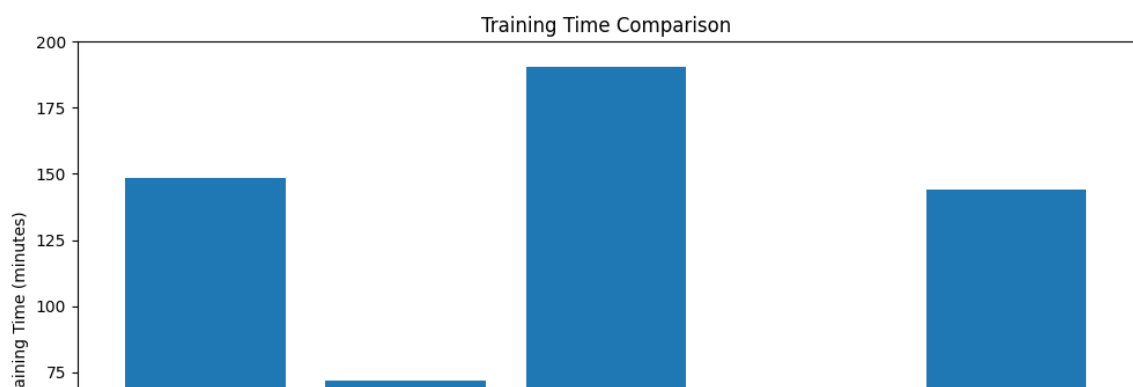
# =====
# STEP 22: TRAINING TIME COMPARISON GRAPH
# =====

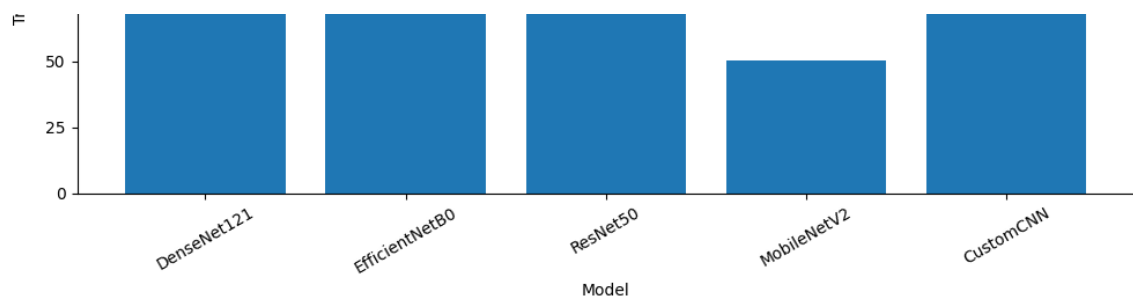
plt.figure(figsize=(10, 6))
plt.bar(results_df["Model"], results_df["Training_Time_min"])
plt.xlabel("Model")
plt.ylabel("Training Time (minutes)")
plt.title("Training Time Comparison")
plt.xticks(rotation=30)
plt.tight_layout()

save_path = FIGURE_DIR / "training_time_comparison.png"
plt.savefig(save_path, dpi=300, bbox_inches="tight")
plt.show()

print("Saved:", save_path)

```





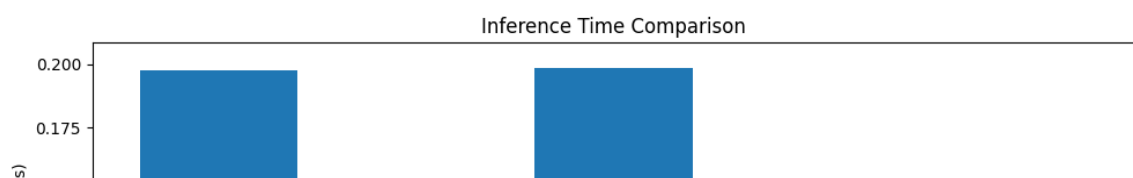
Saved: /content/plantdoc_results/figures/training_time_comparison.png

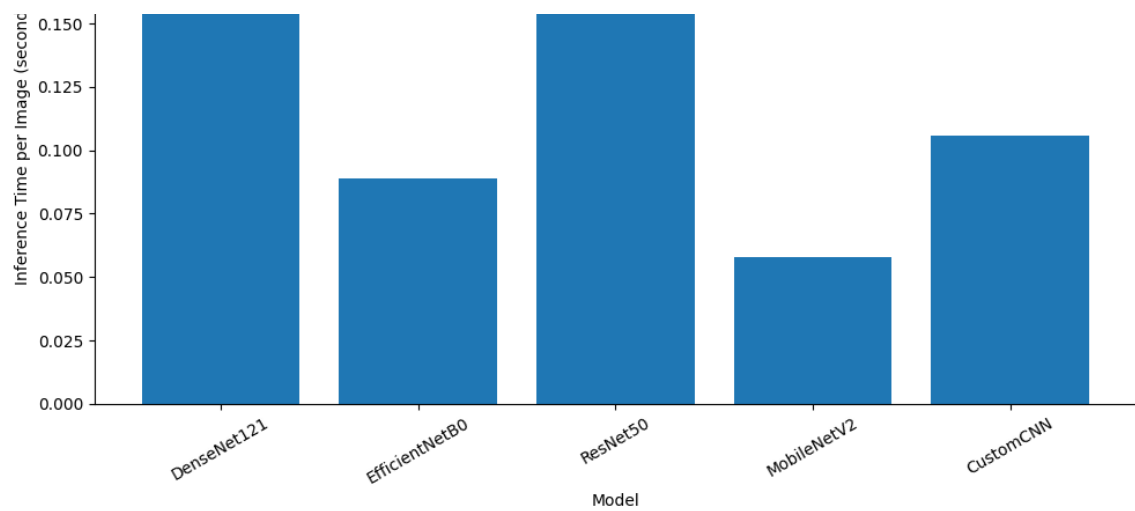
```
# =====
# STEP 23: INFERENCE TIME COMPARISON GRAPH
# =====

plt.figure(figsize=(10, 6))
plt.bar(results_df["Model"], results_df["Inference_Time_per_Image_s"])
plt.xlabel("Model")
plt.ylabel("Inference Time per Image (seconds)")
plt.title("Inference Time Comparison")
plt.xticks(rotation=30)
plt.tight_layout()

save_path = FIGURE_DIR / "inference_time_comparison.png"
plt.savefig(save_path, dpi=300, bbox_inches="tight")
plt.show()

print("Saved:", save_path)
```





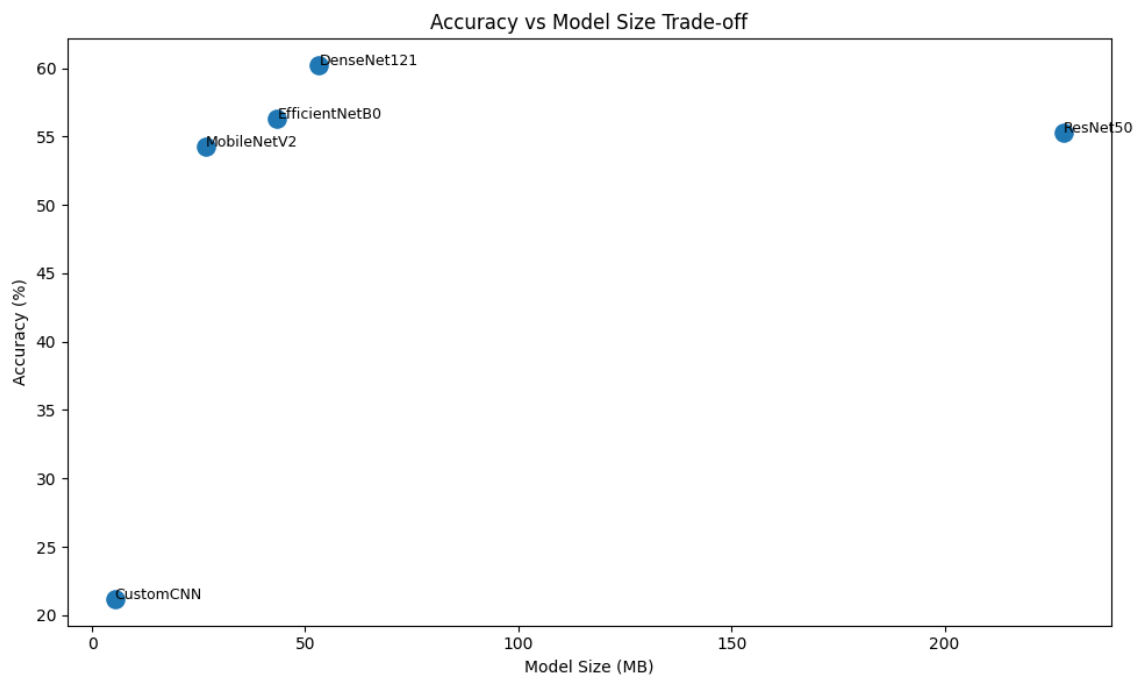
Saved: /content/plantdoc_results/figures/inference_time_comparison.png

```
# =====  
# STEP 24: ACCURACY VS MODEL SIZE TRADE-OFF GRAPH  
# =====  
  
plt.figure(figsize=(10, 6))  
  
plt.scatter(  
    results_df["Model_Size_MB"],  
    results_df["Accuracy"] * 100,  
    s=120  
)  
  
for i, row in results_df.iterrows():  
    plt.text(  
        row["Model_Size_MB"],  
        row["Accuracy"] * 100,  
        row["Model"],  
        fontsize=9  
    )  
  
plt.xlabel("Model Size (MB)")
```

```
plt.ylabel("Accuracy (%)")
plt.title("Accuracy vs Model Size Trade-off")
plt.tight_layout()

save_path = FIGURE_DIR / "accuracy_vs_model_size_tradeoff.png"
plt.savefig(save_path, dpi=300, bbox_inches="tight")
plt.show()

print("Saved:", save_path)
```



Saved: /content/plantdoc_results/figures/accuracy_vs_model_size_tradeoff.png

```
# =====
# STEP 25: F1-SCORE VS INFERENCE TIME TRADE-OFF GRAPH
# =====
```

```
plt.figure(figsize=(10, 6))

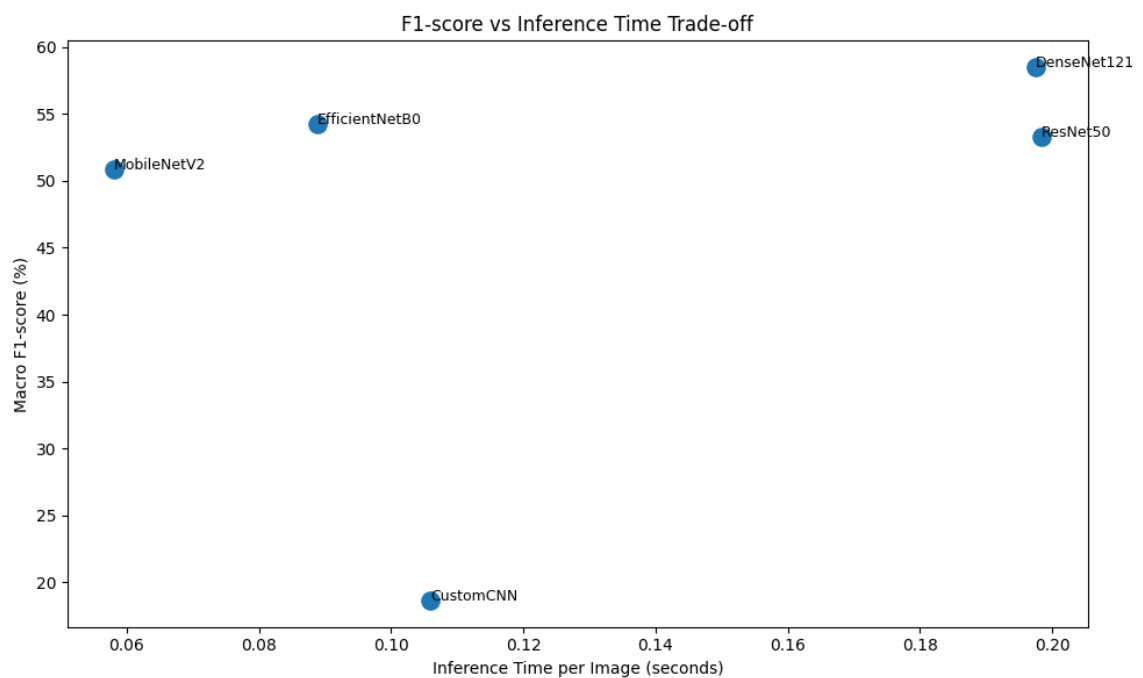
plt.scatter(
    results_df["Inference_Time_per_Image_sec"],
    results_df["F1_Macro"] * 100,
    s=120
)

for i, row in results_df.iterrows():
    plt.text(
        row["Inference_Time_per_Image_sec"],
        row["F1_Macro"] * 100,
        row["Model"],
        fontsize=9
    )

plt.xlabel("Inference Time per Image (seconds)")
plt.ylabel("Macro F1-score (%)")
plt.title("F1-score vs Inference Time Trade-off")
plt.tight_layout()

save_path = FIGURE_DIR / "f1_vs_inference_time_tradeoff.png"
plt.savefig(save_path, dpi=300, bbox_inches="tight")
plt.show()

print("Saved:", save_path)
```



Saved: /content/plantdoc_results/figures/f1_vs_inference_time_tradeoff.png

```
# =====  
# STEP 26: SAVE CLASS-WISE CLASSIFICATION REPORTS  
# =====  
  
for result in all_results:  
    model_name = result["Model"]  
    report_df = pd.DataFrame(result["classification_report"]).trans  
  
    report_path = RESULT_DIR / f"{model_name}_classification_report  
    report_df.to_csv(report_path)  
  
    print("Saved:", report_path)
```

```
Saved: /content/plantdoc_results/CustomCNN_classification_report.csv  
Saved: /content/plantdoc_results/MobileNetV2_classification_report.c  
Saved: /content/plantdoc_results/ResNet50_classification_report.csv  
Saved: /content/plantdoc_results/EfficientNetB0_classification_repor  
Saved: /content/plantdoc_results/DenseNet121_classification_report.c
```

```
# =====  
# STEP 27: IDENTIFY BEST MODELS  
# =====  
  
best_accuracy_model = results_df.sort_values(by="Accuracy", ascendi  
best_f1_model = results_df.sort_values(by="F1_Macro", ascending=Fal  
smallest_model = results_df.sort_values(by="Model_Size_MB", ascendi  
fastest_model = results_df.sort_values(by="Inference_Time_per_Image  
  
print("Best Accuracy Model:")  
print(best_accuracy_model[["Model", "Accuracy"]])  
  
print("\nBest F1-score Model:")
```

```

print(best_f1_model[["Model", "F1_Macro"]])

print("\nSmallest Model:")
print(smallest_model[["Model", "Model_Size_MB"]])

print("\nFastest Inference Model:")
print(fastest_model[["Model", "Inference_Time_per_Image_sec"]])

```

```

Best Accuracy Model:
Model          DenseNet121
Accuracy       0.602067
Name: 0, dtype: object

```

```

Best F1-score Model:
Model          DenseNet121
F1_Macro       0.584723
Name: 0, dtype: object

```

```

Smallest Model:
Model          CustomCNN
Model_Size_MB   5.367273
Name: 4, dtype: object

```

```

Fastest Inference Model:
Model          MobileNetV2
Inference_Time_per_Image_sec  0.058068
Name: 3, dtype: object

```

```

# =====
# STEP 28:  RESULT INTERPRETATION
# =====

```

```

best_acc_name = best_accuracy_model["Model"]
best_acc_value = best_accuracy_model["Accuracy"] * 100

```

```

best_f1_name = best_f1_model["Model"]
best_f1_value = best_f1_model["F1_Macro"] * 100

```

```

small_name = smallest_model["Model"]
small_size = smallest_model["Model_Size_MB"]

```

```

fast_name = fastest_model["Model"]
fast_time = fastest_model["Inference_Time_per_Image_sec"]

```

```

interpretation = f"""
The experimental results show that {best_acc_name} achieved the high
In terms of macro F1-score, {best_f1_name} performed best with an F1

```

```

However, practical deployment requires not only high accuracy but al
The smallest model was {small_name}, with a saved model size of {sma
The fastest model in terms of inference time was {fast_name}, requir

These findings indicate that the best model for accuracy may not alw
Therefore, this comparative analysis helps identify both the most ac
"""

print(interpretation)

text_path = RESULT_DIR / "paper_ready_interpretation.txt"
with open(text_path, "w") as f:
    f.write(interpretation)

print("Saved interpretation:", text_path)

```

The experimental results show that DenseNet121 achieved the highest
 In terms of macro F1-score, DenseNet121 performed best with an F1-sc
 However, practical deployment requires not only high accuracy but al
 The smallest model was CustomCNN, with a saved model size of 5.37 ME
 The fastest model in terms of inference time was MobileNetV2, requir

These findings indicate that the best model for accuracy may not alw
 Therefore, this comparative analysis helps identify both the most ac

Saved interpretation: /content/plantdoc_results/paper_ready_interpre

```

# =====
# STEP 29: ZIP ALL RESULTS FOR DOWNLOAD
# =====

zip_output = "/content/plantdoc_5_model_results.zip"

if os.path.exists(zip_output):
    os.remove(zip_output)

shutil.make_archive(
    base_name="/content/plantdoc_5_model_results",
    format="zip",
    root_dir=str(RESULT_DIR)
)

print("ZIP file created:", zip_output)

```


ZIP file created: /content/plantdoc_5_model_results.zip

```
# =====  
# STEP 30: DOWNLOAD RESULT ZIP  
# =====  
  
from google.colab import files  
  
files.download("/content/plantdoc_5_model_results.zip")
```

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.